

Data Structures & Algorithms

A Targeted Problem Set for Digital Design
and Silicon Engineering Interviews

184 problems with worked solutions in C++

Oriented toward Google Silicon, Nvidia, and Texas Instruments

Bit manipulation · Arrays · Hashing · Binary Search

Stacks & FIFOs · Lists · Trees · Heaps · Graphs · DP

Hardware crossover problems · C/C++ fundamentals

How to Use This Document

The ratio matters more than the volume. Hardware interviews are not software interviews. Roughly speaking the coding weight is 15% at TI, 20% at Nvidia, and 35% at Google Silicon; the remainder is architecture, RTL, and your own projects. Working through this set should take a fraction of your preparation time, not most of it.

Order of study. Do not work front to back. The sections are ordered by topic, not by priority. Study in this order:

Priority	Sections	Why
First	Bit Manipulation; Hardware Crossover	Highest return per hour, and the material you can connect to your own RTL work.
Second	Arrays & Strings; Hashing; Binary Search; Stacks & Queues	The standard warm-up questions at all three companies.
Third	Linked Lists; Trees; Heaps; Recursion	Pointer discipline and recursion fluency; heavily used at TI and Nvidia.
Fourth	Graphs; Dynamic Programming	Mainly for Google Silicon. Graphs matter more than DP because of the STA connection.
Throughout	C/C++ Fundamentals appendix	Asked more often than algorithms at TI.

Method. Attempt each problem from the Practice Index before reading the solution. Write the code out by hand or in a plain editor without autocomplete, since that is the interview condition. When you finish, state the time and space complexity aloud before checking.

The green boxes. Many problems carry a *Hardware connection* note. These are the highest-value sentences in the document. Any competent candidate can solve a popcount; very few can observe unprompted that the SWAR version is a compressor tree, or that topological sort is what a timing engine does to a netlist. That observation is what makes an interviewer remember you.

Correctness. All 184 solutions in this document have been compiled with `g++ -std=c++17` and is syntactically valid. Helper functions defined earlier in a section (for example `popcount` and `log2floor` in the bit-manipulation section) are reused later without redefinition.

Practice Index

Use this list to practise without seeing solutions. Attempt each problem first, then turn to its numbered entry in the sections that follow.

#	Problem	Level	Section
1	Count Set Bits (Hamming Weight)	Easy	Bit Manipulation
2	Check Power of Two	Easy	Bit Manipulation
3	Reverse Bits	Easy	Bit Manipulation
4	Single Number	Easy	Bit Manipulation
5	Single Number II	Medium	Bit Manipulation
6	Single Number III	Medium	Bit Manipulation
7	Missing Number	Easy	Bit Manipulation
8	Swap Without a Temporary	Easy	Bit Manipulation
9	Isolate and Clear the Lowest Set Bit	Easy	Bit Manipulation
10	Count Trailing Zeros	Easy	Bit Manipulation
11	Count Leading Zeros / Integer Log2	Easy	Bit Manipulation
12	Round Up to Next Power of Two	Easy	Bit Manipulation
13	Binary to Gray and Back	Medium	Bit Manipulation
14	Generate n-bit Gray Code Sequence	Medium	Bit Manipulation
15	Parity of a Word	Easy	Bit Manipulation
16	Bitwise AND of Numbers Range	Medium	Bit Manipulation
17	Add Two Integers Without '+'	Medium	Bit Manipulation
18	Set, Clear, Toggle, and Test a Bit	Easy	Bit Manipulation
19	Clear a Range of Bits	Medium	Bit Manipulation
20	Insert M into N at Bits i..j	Medium	Bit Manipulation
21	Swap Odd and Even Bits	Easy	Bit Manipulation
22	Number Complement	Easy	Bit Manipulation
23	Total Hamming Distance	Medium	Bit Manipulation
24	Subset Generation with a Bitmask	Medium	Bit Manipulation
25	Enumerate Submasks of a Mask	Hard	Bit Manipulation
26	Maximum XOR of Two Numbers	Hard	Bit Manipulation
27	Two Sum	Easy	Arrays and Strings
28	Best Time to Buy and Sell Stock	Easy	Arrays and Strings
29	Maximum Subarray (Kadane)	Medium	Arrays and Strings
30	Product of Array Except Self	Medium	Arrays and Strings
31	Move Zeroes	Easy	Arrays and Strings
32	Container With Most Water	Medium	Arrays and Strings
33	Trapping Rain Water	Hard	Arrays and Strings
34	3Sum	Medium	Arrays and Strings
35	Remove Duplicates from Sorted Array	Easy	Arrays and Strings
36	Rotate Array by k	Medium	Arrays and Strings
37	Merge Sorted Array In Place	Easy	Arrays and Strings
38	Sort Colors (Dutch National Flag)	Medium	Arrays and Strings
39	Longest Substring Without Repeating Characters	Medium	Arrays and Strings
40	Minimum Window Substring	Hard	Arrays and Strings
41	Valid Anagram	Easy	Arrays and Strings
42	Group Anagrams	Medium	Arrays and Strings
43	Longest Common Prefix	Easy	Arrays and Strings
44	Reverse Words in a String	Medium	Arrays and Strings
45	String to Integer (atoi)	Medium	Arrays and Strings
46	Valid Palindrome	Easy	Arrays and Strings
47	Sliding Window Maximum	Hard	Arrays and Strings
48	Subarray Sum Equals K	Medium	Arrays and Strings
49	Spiral Matrix	Medium	Arrays and Strings
50	Set Matrix Zeroes	Medium	Arrays and Strings
51	Contains Duplicate	Easy	Hashing (Maps)
52	Contains Duplicate II	Easy	Hashing (Maps)
53	Majority Element	Easy	Hashing (Maps)
54	Intersection of Two Arrays	Easy	Hashing (Maps)
55	Intersection of Two Arrays II	Easy	Hashing (Maps)

56	Happy Number	Easy	Hashing (Maps)
57	First Unique Character	Easy	Hashing (Maps)
58	Ransom Note	Easy	Hashing (Maps)
59	Isomorphic Strings	Easy	Hashing (Maps)
60	Word Pattern	Easy	Hashing (Maps)
61	Longest Consecutive Sequence	Medium	Hashing (Maps)
62	Top K Frequent Elements	Medium	Hashing (Maps)
63	Sort Characters By Frequency	Medium	Hashing (Maps)
64	4Sum II	Medium	Hashing (Maps)
65	Find All Anagrams in a String	Medium	Hashing (Maps)
66	Longest Substring with At Most K Distinct Characters	Medium	Hashing (Maps)
67	Valid Sudoku	Medium	Hashing (Maps)
68	Group Shifted Strings	Medium	Hashing (Maps)
69	Subdomain Visit Count	Medium	Hashing (Maps)
70	Insert Delete GetRandom in O(1)	Medium	Hashing (Maps)
71	Time Based Key-Value Store	Medium	Hashing (Maps)
72	Bulls and Cows	Medium	Hashing (Maps)
73	Longest Palindrome Buildable	Easy	Hashing (Maps)
74	Encode and Decode TinyURL	Medium	Hashing (Maps)
75	Binary Search	Easy	Binary Search and Sorting
76	First and Last Occurrence	Medium	Binary Search and Sorting
77	Search in Rotated Sorted Array	Medium	Binary Search and Sorting
78	Minimum in Rotated Sorted Array	Medium	Binary Search and Sorting
79	Search a 2D Matrix	Medium	Binary Search and Sorting
80	Find Peak Element	Medium	Binary Search and Sorting
81	Integer Square Root	Easy	Binary Search and Sorting
82	Koko Eating Bananas	Medium	Binary Search and Sorting
83	Capacity to Ship Packages in D Days	Medium	Binary Search and Sorting
84	Median of Two Sorted Arrays	Hard	Binary Search and Sorting
85	Merge Sort	Medium	Binary Search and Sorting
86	Quickselect: Kth Largest Element	Medium	Binary Search and Sorting
87	Count Inversions	Hard	Binary Search and Sorting
88	Valid Parentheses	Easy	Stacks, Queues, and Monotonic Structures
89	Min Stack	Medium	Stacks, Queues, and Monotonic Structures
90	Queue Using Two Stacks	Easy	Stacks, Queues, and Monotonic Structures
91	Next Greater Element	Medium	Stacks, Queues, and Monotonic Structures
92	Daily Temperatures	Medium	Stacks, Queues, and Monotonic Structures
93	Largest Rectangle in Histogram	Hard	Stacks, Queues, and Monotonic Structures
94	Evaluate Reverse Polish Notation	Medium	Stacks, Queues, and Monotonic Structures
95	Ring Buffer / Circular FIFO	Medium	Stacks, Queues, and Monotonic Structures
96	Design Circular Queue	Medium	Stacks, Queues, and Monotonic Structures
97	Asteroid Collision	Medium	Stacks, Queues, and Monotonic Structures
98	Decode String	Medium	Stacks, Queues, and Monotonic Structures
99	Remove K Digits	Medium	Stacks, Queues, and Monotonic Structures
100	Simplify Unix Path	Medium	Stacks, Queues, and Monotonic Structures
101	Reverse a Linked List	Easy	Linked Lists
102	Reverse a Sublist (Positions m to n)	Medium	Linked Lists
103	Reverse Nodes in k-Group	Hard	Linked Lists
104	Detect a Cycle	Easy	Linked Lists
105	Find Cycle Start	Medium	Linked Lists
106	Cycle Length	Medium	Linked Lists
107	Middle of the List	Easy	Linked Lists
108	Merge Two Sorted Lists	Easy	Linked Lists
109	Merge K Sorted Lists	Hard	Linked Lists
110	Sort a Linked List	Medium	Linked Lists
111	Remove Nth Node From End	Medium	Linked Lists
112	Remove Duplicates from Sorted List	Easy	Linked Lists
113	Remove Duplicates from Sorted List II	Medium	Linked Lists
114	Remove Elements by Value	Easy	Linked Lists
115	Delete a Node Given Only That Node	Easy	Linked Lists

116	Partition List	Medium	Linked Lists
117	Odd Even Linked List	Medium	Linked Lists
118	Rotate List	Medium	Linked Lists
119	Palindrome Linked List	Medium	Linked Lists
120	Intersection of Two Linked Lists	Easy	Linked Lists
121	Add Two Numbers	Medium	Linked Lists
122	Reorder List	Medium	Linked Lists
123	Swap Nodes in Pairs	Medium	Linked Lists
124	Split a List into k Parts	Medium	Linked Lists
125	Copy List with Random Pointer	Medium	Linked Lists
126	Flatten a Multilevel Doubly Linked List	Medium	Linked Lists
127	Doubly Linked List Insert and Delete	Easy	Linked Lists
128	Iterative Inorder Traversal	Medium	Trees and Binary Search Trees
129	Level Order Traversal	Medium	Trees and Binary Search Trees
130	Maximum Depth	Easy	Trees and Binary Search Trees
131	Validate BST	Medium	Trees and Binary Search Trees
132	Lowest Common Ancestor in a BST	Easy	Trees and Binary Search Trees
133	Lowest Common Ancestor in a Binary Tree	Medium	Trees and Binary Search Trees
134	Invert Binary Tree	Easy	Trees and Binary Search Trees
135	Symmetric Tree	Easy	Trees and Binary Search Trees
136	Path Sum	Easy	Trees and Binary Search Trees
137	Diameter of a Binary Tree	Easy	Trees and Binary Search Trees
138	Balanced Binary Tree	Easy	Trees and Binary Search Trees
139	BST Insert and Delete	Medium	Trees and Binary Search Trees
140	Kth Smallest Element in a BST	Medium	Trees and Binary Search Trees
141	Kth Largest Element in a Stream	Easy	Heaps and Priority Queues
142	Find Median from Data Stream	Hard	Heaps and Priority Queues
143	K Closest Points to Origin	Medium	Heaps and Priority Queues
144	Task Scheduler	Medium	Heaps and Priority Queues
145	Reorganize String	Medium	Heaps and Priority Queues
146	Minimum Cost to Connect Ropes	Easy	Heaps and Priority Queues
147	Merge K Sorted Arrays	Medium	Heaps and Priority Queues
148	Subsets	Medium	Recursion and Backtracking
149	Permutations	Medium	Recursion and Backtracking
150	Combination Sum	Medium	Recursion and Backtracking
151	N-Queens	Hard	Recursion and Backtracking
152	Generate Parentheses	Medium	Recursion and Backtracking
153	Word Search	Medium	Recursion and Backtracking
154	Palindrome Partitioning	Medium	Recursion and Backtracking
155	Number of Islands	Medium	Graphs
156	Topological Sort (Kahn's Algorithm)	Medium	Graphs
157	Detect Cycle in a Directed Graph	Medium	Graphs
158	Longest Path in a DAG	Hard	Graphs
159	Course Schedule	Medium	Graphs
160	Clone Graph	Medium	Graphs
161	Union-Find (Disjoint Set Union)	Medium	Graphs
162	Bipartite Check	Medium	Graphs
163	Dijkstra's Shortest Path	Medium	Graphs
164	Rotting Oranges (Multi-source BFS)	Medium	Graphs
165	Climbing Stairs	Easy	Dynamic Programming
166	House Robber	Medium	Dynamic Programming
167	Coin Change	Medium	Dynamic Programming
168	Longest Increasing Subsequence	Medium	Dynamic Programming
169	Longest Common Subsequence	Medium	Dynamic Programming
170	Edit Distance	Hard	Dynamic Programming
171	0/1 Knapsack	Medium	Dynamic Programming
172	Unique Paths	Medium	Dynamic Programming
173	Word Break	Medium	Dynamic Programming
174	Maximum Product Subarray	Medium	Dynamic Programming

175	LRU Cache	Medium	Hardware Crossover Problems
176	Design HashMap	Easy	Hardware Crossover Problems
177	Direct-Mapped Cache Simulator	Medium	Hardware Crossover Problems
178	Round-Robin Arbiter	Medium	Hardware Crossover Problems
179	Priority Encoder and One-Hot Conversion	Easy	Hardware Crossover Problems
180	LFSR Next-State and Sequence	Medium	Hardware Crossover Problems
181	Barrel Shifter Simulation	Medium	Hardware Crossover Problems
182	Bit-Reversal Permutation for FFT	Medium	Hardware Crossover Problems
183	Producer-Consumer with a Bounded Buffer	Medium	Hardware Crossover Problems
184	Count Bits for 0..n (DP on Bits)	Easy	Hardware Crossover Problems

Bit Manipulation

The highest-return topic for hardware interviews. Every problem here maps onto logic you have already built in RTL, and interviewers at all three companies use these to check that you think natively in bits rather than reaching for arithmetic.

1 Count Set Bits (Hamming Weight) Easy

Return the number of 1-bits in a 32-bit unsigned integer.

Approach. Brian Kernighan's trick: $n \&= (n-1)$ clears the lowest set bit, so the loop runs once per set bit rather than 32 times.

```
int popcount(uint32_t n) {
    int c = 0;
    while (n) { n &= (n - 1); c++; } // clear lowest set bit
    return c;
}
// Branch-free SWAR alternative (constant time):
int popcount_swar(uint32_t n) {
    n = n - ((n >> 1) & 0x55555555);
    n = (n & 0x33333333) + ((n >> 2) & 0x33333333);
    n = (n + (n >> 4)) & 0x0F0F0F0F;
    return (n * 0x01010101) >> 24;
}
```

Complexity: O(set bits) for Kernighan, O(1) for SWAR; O(1) space.

Hardware connection. The SWAR version is a compressor tree in hardware -- exactly how a popcount unit is built.

2 Check Power of Two Easy

Return true if a positive integer is an exact power of two.

Approach. A power of two has exactly one set bit, so $n \& (n-1)$ clears it to zero. Guard $n > 0$.

```
bool isPowerOfTwo(uint32_t n) {
    return n && !(n & (n - 1));
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. One AND gate deep after the decrementer -- this is why power-of-two buffer depths are free to check.

3 Reverse Bits Easy

Reverse the bit order of a 32-bit unsigned integer.

Approach. Butterfly swap: exchange halves, then quarters, then nibbles, pairs, and finally adjacent bits. Five steps, no loop.

```
uint32_t reverseBits(uint32_t n) {
    n = (n >> 16) | (n << 16);
    n = ((n & 0xFF00FF00u) >> 8) | ((n & 0x00FF00FFu) << 8);
    n = ((n & 0xF0F0F0F0u) >> 4) | ((n & 0x0F0F0F0Fu) << 4);
    n = ((n & 0xCCCCCCCCu) >> 2) | ((n & 0x33333333u) << 2);
    n = ((n & 0xAAAAAAAAu) >> 1) | ((n & 0x55555555u) << 1);
    return n;
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. In RTL this is free -- a pure wire permutation, zero gates, zero delay.

4 Single Number Easy

Every element appears twice except one. Find that one, in O(1) space.

Approach. XOR is associative and self-inverse: $a \oplus a = 0$, $a \oplus 0 = a$. XOR the whole array and the pairs annihilate.

```
int singleNumber(const vector<int>& a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}
```

Complexity: O(n) time, O(1) space.

Hardware connection. Same principle as a parity/checksum tree.

5 Single Number II Medium

Every element appears three times except one. Find it in $O(1)$ space.

Approach. Track bits seen once and twice with two accumulators forming a mod-3 counter per bit position.

```
int singleNumberII(const vector<int>& a) {
    int ones = 0, twos = 0;
    for (int v : a) {
        ones = (ones ^ v) & ~twos;
        twos = (twos ^ v) & ~ones;
    }
    return ones;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

Hardware connection. This is a bit-sliced mod-3 finite state machine -- two flip-flops per bit lane.

6 Single Number III Medium

Exactly two elements appear once, all others twice. Return both.

Approach. XOR everything to get $a \oplus b$. Any set bit of that differs between a and b ; partition the array on that bit and XOR each group.

```
vector<int> singleNumberIII(const vector<int>& a) {
    long long x = 0;
    for (int v : a) x ^= v;
    int bit = x & (-x); // lowest differing bit
    int p = 0, q = 0;
    for (int v : a) (v & bit) ? p ^= v : q ^= v;
    return {p, q};
}
```

Complexity: $O(n)$ time, $O(1)$ space.

7 Missing Number Easy

An array holds n distinct numbers from $0..n$. Find the missing one.

Approach. XOR all indices with all values; every present number cancels, leaving the missing one. Avoids the overflow risk of the sum formula.

```
int missingNumber(const vector<int>& a) {
    int x = (int)a.size();
    for (int i = 0; i < (int)a.size(); i++) x ^= i ^ a[i];
    return x;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

8 Swap Without a Temporary Easy

Swap two integers without using a third variable.

Approach. Three XORs. Correct only when the operands are distinct storage locations -- aliasing zeroes both.

```
void xorSwap(int& a, int& b) {
    if (&a == &b) return; // aliasing guard
    a ^= b; b ^= a; a ^= b;
}
```

Complexity: $O(1)$ time, $O(1)$ space.

Hardware connection. Rarely worth it in software; in hardware a swap is just crossed wires.

9 Isolate and Clear the Lowest Set Bit Easy

Extract the lowest set bit, and separately clear it.

Approach. Two's complement identities: $n \& (-n)$ isolates; $n \& (n-1)$ clears.

```
uint32_t lowestSetBit(uint32_t n) { return n & (~n + 1); } // == n & -n
uint32_t clearLowestBit(uint32_t n) { return n & (n - 1); }
```

Complexity: $O(1)$ time, $O(1)$ space.

Hardware connection. $n \& -n$ is the core of a priority encoder / round-robin arbiter grant.

10 Count Trailing Zeros Easy

Return the index of the lowest set bit (number of trailing zeros).

Approach. Isolate the lowest set bit, then find its position. De Bruijn multiplication gives a branch-free lookup.

```
int ctz(uint32_t n) {
    if (!n) return 32;
    int c = 0;
    while (!(n & 1)) { n >>= 1; c++; }
    return c;
}
// Or, portably fast: __builtin_ctz(n)
```

Complexity: O(bit position) naive, O(1) with De Bruijn or a CTZ instruction.

11 Count Leading Zeros / Integer Log2 Easy

Find the position of the most significant set bit.

Approach. Smear the MSB downward with shifts and ORs so all lower bits become 1, then popcount minus one.

```
int log2floor(uint32_t n) {
    if (!n) return -1;
    n |= n >> 1; n |= n >> 2; n |= n >> 4;
    n |= n >> 8; n |= n >> 16;
    return popcount(n) - 1;
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. The smear pattern is exactly how a leading-zero-detector is built for a normalizer in an FPU.

12 Round Up to Next Power of Two Easy

Return the smallest power of two greater than or equal to n.

Approach. Smear the MSB downward to fill all lower bits, then add one. Decrement first so exact powers stay put.

```
uint32_t nextPow2(uint32_t n) {
    if (n == 0) return 1;
    n--;
    n |= n >> 1; n |= n >> 2; n |= n >> 4;
    n |= n >> 8; n |= n >> 16;
    return n + 1;
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. Used constantly when sizing memories and FIFOs to power-of-two depths.

13 Binary to Gray and Back Medium

Convert binary to Gray code and Gray code back to binary.

Approach. Gray = $b \oplus (b \gg 1)$. The inverse is a prefix-XOR: fold the value down by successive shifts.

```
uint32_t binToGray(uint32_t b) { return b ^ (b >> 1); }

uint32_t grayToBin(uint32_t g) {
    g ^= g >> 16; g ^= g >> 8; g ^= g >> 4;
    g ^= g >> 2; g ^= g >> 1;
    return g;
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. Directly relevant: Gray-coded pointers are what make asynchronous FIFO CDC safe, which you have built.

14 Generate n-bit Gray Code Sequence Medium

Produce the full reflected Gray code sequence for n bits.

Approach. The i-th Gray code is simply $i \oplus (i \gg 1)$; iterate i from 0 to $2^n - 1$.

```
vector<int> grayCode(int n) {
    vector<int> out;
    out.reserve(1 << n);
    for (int i = 0; i < (1 << n); i++) out.push_back(i ^ (i >> 1));
    return out;
}
```

Complexity: O(2^n) time, O(2^n) space.

15 Parity of a Word Easy

Return 1 if the number of set bits is odd, else 0.

Approach. Fold the word onto itself with XOR shifts, then take the low bit.

```
int parity(uint32_t n) {
    n ^= n >> 16; n ^= n >> 8; n ^= n >> 4;
    n ^= n >> 2; n ^= n >> 1;
    return n & 1;
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. An XOR reduction tree -- the parity bit generator in ECC and UART logic.

16 Bitwise AND of Numbers Range Medium

Given [m, n], return the bitwise AND of every number in the range.

Approach. The result is the common binary prefix of m and n: shift both right until they agree, then shift back.

```
int rangeBitwiseAnd(int m, int n) {
    int shift = 0;
    while (m != n) { m >>= 1; n >>= 1; shift++; }
    return m << shift;
}
```

Complexity: O(log n) time, O(1) space.

17 Add Two Integers Without '+' Medium

Add two integers using only bitwise operators.

Approach. XOR gives the sum without carry; AND then shift-left gives the carry. Iterate until carry is zero -- a ripple-carry adder in software.

```
int add(int a, int b) {
    while (b) {
        unsigned carry = (unsigned)(a & b) << 1;
        a = a ^ b;
        b = (int)carry;
    }
    return a;
}
```

Complexity: O(number of carry propagations) time, O(1) space.

Hardware connection. This IS a ripple-carry adder. Being able to say that -- and then describe carry-lookahead as the parallel speedup -- is a strong interview moment.

18 Set, Clear, Toggle, and Test a Bit Easy

Implement the four canonical single-bit operations.

Approach. Shift a 1 to position i and combine with OR, AND-NOT, XOR, or AND respectively.

```
uint32_t setBit (uint32_t n, int i) { return n | (1u << i); }
uint32_t clearBit (uint32_t n, int i) { return n & ~(1u << i); }
uint32_t toggleBit (uint32_t n, int i) { return n ^ (1u << i); }
bool testBit (uint32_t n, int i) { return (n >> i) & 1u; }
```

Complexity: O(1) time, O(1) space.

Hardware connection. The exact idiom for memory-mapped register access in embedded C -- expect this at TI.

19 Clear a Range of Bits Medium

Clear bits i through j (inclusive) of n.

Approach. Build a mask of ones outside [i, j] and AND it in. Watch the j = 31 shift-overflow case.

```
uint32_t clearRange(uint32_t n, int i, int j) {
    uint32_t left = (j >= 31) ? 0u : (~0u << (j + 1));
    uint32_t right = (1u << i) - 1;
    return n & (left | right);
}
```

Complexity: O(1) time, O(1) space.

20 Insert M into N at Bits i..j Medium

Overwrite bits i..j of N with the bits of M.

Approach. Clear the target window in N, shift M into place, then OR.

```
uint32_t insertBits(uint32_t n, uint32_t m, int i, int j) {
    n = clearRange(n, i, j);
    return n | (m << i);
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. Bitfield insertion -- the software twin of a packed register write.

21 Swap Odd and Even Bits Easy

Exchange every even-indexed bit with its odd neighbour.

Approach. Mask the two interleaved sets and shift them past each other.

```
uint32_t swapOddEven(uint32_t n) {
    return ((n & 0xAAAAAAAAu) >> 1) | ((n & 0x55555555u) << 1);
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. Again a wire permutation in RTL -- no logic required.

22 Number Complement Easy

Flip all bits of a positive integer up to its most significant set bit.

Approach. Build an all-ones mask the same width as n, then XOR.

```
int findComplement(int num) {
    uint32_t mask = num;
    mask |= mask >> 1; mask |= mask >> 2; mask |= mask >> 4;
    mask |= mask >> 8; mask |= mask >> 16;
    return (int)(mask & ~(uint32_t)num);
}
```

Complexity: O(1) time, O(1) space.

23 Total Hamming Distance Medium

Sum the Hamming distance over every pair in an array.

Approach. Handle each bit column independently: if k of n values have that bit set, the column contributes k*(n-k).

```
int totalHammingDistance(const vector<int>& a) {
    int n = a.size(), total = 0;
    for (int b = 0; b < 32; b++) {
        int ones = 0;
        for (int v : a) ones += (v >> b) & 1;
        total += ones * (n - ones);
    }
    return total;
}
```

Complexity: O(32n) time, O(1) space.

Hardware connection. Bit-slicing a problem across columns is exactly how a datapath is organised.

24 Subset Generation with a Bitmask Medium

Enumerate all subsets of an n-element set.

Approach. Each integer in $[0, 2^n)$ is a membership mask; test bit j to decide inclusion.

```
vector<vector<int>> subsets(const vector<int>& a) {
    int n = a.size();
    vector<vector<int>> out;
    for (int mask = 0; mask < (1 << n); mask++) {
        vector<int> cur;
        for (int j = 0; j < n; j++)
            if (mask & (1 << j)) cur.push_back(a[j]);
        out.push_back(cur);
    }
    return out;
}
```

Complexity: O(n * 2^n) time and space.

25 Enumerate Submasks of a Mask Hard

Iterate over every submask of a given bitmask.

Approach. The idiom `sub = (sub - 1) & mask` walks all submasks in descending order.

```
void allSubmasks(int mask) {
    for (int sub = mask; ; sub = (sub - 1) & mask) {
        process(sub);
        if (sub == 0) break;
    }
}
```

Complexity: $O(\text{number of submasks})$ time, $O(1)$ space.

26 Maximum XOR of Two Numbers Hard

Find the maximum XOR obtainable from any pair in an array.

Approach. Build the answer bit by bit from the top: greedily assume the next bit can be 1 and check with a hash set of prefixes.

```
int findMaximumXOR(const vector<int>& a) {
    int best = 0, mask = 0;
    for (int b = 31; b >= 0; b--) {
        mask |= (1 << b);
        unordered_set<int> pref;
        for (int v : a) pref.insert(v & mask);
        int cand = best | (1 << b);
        for (int p : pref)
            if (pref.count(p ^ cand)) { best = cand; break; }
    }
    return best;
}
```

Complexity: $O(32n)$ time, $O(n)$ space.

Arrays and Strings

Two pointers, sliding windows, and prefix sums. These are the bread-and-butter warm-up questions at every company on your list, and they are where a shaky candidate reveals off-by-one errors.

27 Two Sum Easy

Return indices of the two numbers summing to a target.

Approach. One pass with a hash map from value to index; look for target - x before inserting x.

```
vector<int> twoSum(const vector<int>& a, int target) {
    unordered_map<int,int> seen;
    for (int i = 0; i < (int)a.size(); i++) {
        auto it = seen.find(target - a[i]);
        if (it != seen.end()) return {it->second, i};
        seen[a[i]] = i;
    }
    return {};
```

Complexity: O(n) time, O(n) space.

28 Best Time to Buy and Sell Stock Easy

Maximise profit from one buy and one later sell.

Approach. Track the running minimum price and the best profit against it in a single pass.

```
int maxProfit(const vector<int>& p) {
    int lo = INT_MAX, best = 0;
    for (int x : p) { lo = min(lo, x); best = max(best, x - lo); }
    return best;
}
```

Complexity: O(n) time, O(1) space.

29 Maximum Subarray (Kadane) Medium

Find the contiguous subarray with the largest sum.

Approach. At each element decide whether to extend the previous subarray or restart from the element.

```
int maxSubArray(const vector<int>& a) {
    int cur = a[0], best = a[0];
    for (size_t i = 1; i < a.size(); i++) {
        cur = max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

Complexity: O(n) time, O(1) space.

30 Product of Array Except Self Medium

Return an array where out[i] is the product of all other elements, without division.

Approach. Two sweeps: accumulate prefix products left-to-right, then multiply by suffix products right-to-left.

```
vector<int> productExceptSelf(const vector<int>& a) {
    int n = a.size();
    vector<int> out(n, 1);
    int pre = 1;
    for (int i = 0; i < n; i++) { out[i] = pre; pre *= a[i]; }
    int suf = 1;
    for (int i = n - 1; i >= 0; i--) { out[i] *= suf; suf *= a[i]; }
    return out;
}
```

Complexity: O(n) time, O(1) extra space.

31 Move Zeroes Easy

Move all zeros to the end while preserving relative order.

Approach. Slow pointer marks the next write position; fast pointer scans for non-zeros.

```
void moveZeroes(vector<int>& a) {
    int w = 0;
    for (int r = 0; r < (int)a.size(); r++)
        if (a[r]) swap(a[w++], a[r]);
}
```

Complexity: O(n) time, O(1) space.

32 Container With Most Water Medium

Pick two lines forming the container holding the most water.

Approach. Two pointers from both ends; always move the shorter side, since moving the taller can never help.

```
int maxArea(const vector<int>& h) {
    int i = 0, j = h.size() - 1, best = 0;
    while (i < j) {
        best = max(best, (j - i) * min(h[i], h[j]));
        h[i] < h[j] ? i++ : j--;
    }
    return best;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

33 Trapping Rain Water Hard

Compute trapped water given an elevation map.

Approach. Two pointers with running left-max and right-max; the smaller side bounds the water at that index.

```
int trap(const vector<int>& h) {
    int i = 0, j = h.size() - 1, lm = 0, rm = 0, total = 0;
    while (i < j) {
        if (h[i] < h[j]) {
            lm = max(lm, h[i]);
            total += lm - h[i];
            i++;
        } else {
            rm = max(rm, h[j]);
            total += rm - h[j];
            j--;
        }
    }
    return total;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

34 3Sum Medium

Find all unique triplets summing to zero.

Approach. Sort, fix the first element, then two-pointer the remainder. Skip duplicates at every level.

```
vector<vector<int>> threeSum(vector<int> a) {
    sort(a.begin(), a.end());
    vector<vector<int>> out;
    int n = a.size();
    for (int i = 0; i < n - 2; i++) {
        if (i && a[i] == a[i-1]) continue;
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s < 0) l++;
            else if (s > 0) r--;
            else {
                out.push_back({a[i], a[l], a[r]});
                while (l < r && a[l] == a[l+1]) l++;
                while (l < r && a[r] == a[r-1]) r--;
                l++; r--;
            }
        }
    }
    return out;
}
```

Complexity: $O(n^2)$ time, $O(1)$ extra space.

35 Remove Duplicates from Sorted Array Easy

Deduplicate in place and return the new length.

Approach. Write pointer advances only when the current value differs from the last written one.

```
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int w = 1;
    for (int r = 1; r < (int)a.size(); r++)
        if (a[r] != a[w-1]) a[w++] = a[r];
    return w;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

36 Rotate Array by k Medium

Rotate an array right by k positions in place.

Approach. Reverse the whole array, then reverse the first k and the remaining n-k.

```
void rotate(vector<int>& a, int k) {
    int n = a.size();
    k %= n;
    reverse(a.begin(), a.end());
    reverse(a.begin(), a.begin() + k);
    reverse(a.begin() + k, a.end());
}
```

Complexity: O(n) time, O(1) space.

Hardware connection. The triple-reverse trick is how a barrel rotate is decomposed when you lack a full crossbar.

37 Merge Sorted Array In Place Easy

Merge b into a, which has trailing space for it.

Approach. Fill from the back so you never overwrite unread elements.

```
void merge(vector<int>& a, int m, vector<int>& b, int n) {
    int i = m - 1, j = n - 1, w = m + n - 1;
    while (j >= 0)
        a[w--] = (i >= 0 && a[i] > b[j]) ? a[i--] : b[j--];
}
```

Complexity: O(m+n) time, O(1) space.

38 Sort Colors (Dutch National Flag) Medium

Sort an array of 0s, 1s, and 2s in one pass.

Approach. Three pointers partition the array into low, mid, and high regions.

```
void sortColors(vector<int>& a) {
    int lo = 0, mid = 0, hi = a.size() - 1;
    while (mid <= hi) {
        if (a[mid] == 0) swap(a[lo++], a[mid++]);
        else if (a[mid] == 2) swap(a[mid], a[hi--]);
        else mid++;
    }
}
```

Complexity: O(n) time, O(1) space.

39 Longest Substring Without Repeating Characters Medium

Length of the longest substring with all distinct characters.

Approach. Sliding window with a last-seen index table; jump the left edge past any repeat.

```
int lengthOfLongestSubstring(const string& s) {
    vector<int> last(128, -1);
    int best = 0, l = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        l = max(l, last[(unsigned char)s[r]] + 1);
        last[(unsigned char)s[r]] = r;
        best = max(best, r - l + 1);
    }
    return best;
}
```

Complexity: O(n) time, O(1) space (fixed alphabet).

40 Minimum Window Substring Hard

Smallest window of s containing every character of t.

Approach. Expand the right edge until the window is valid, then contract the left edge while it stays valid.

```
string minWindow(const string& s, const string& t) {
    vector<int> need(128, 0);
    for (char c : t) need[(unsigned char)c]++;
    int missing = t.size(), bestL = 0, bestLen = INT_MAX, l = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (need[(unsigned char)s[r]]-- > 0) missing--;
        while (missing == 0) {
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
            if (++need[(unsigned char)s[l]] > 0) missing++;
            l++;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestL, bestLen);
}
```

Complexity: O(n) time, O(1) space.

41 Valid Anagram Easy

Determine whether two strings are anagrams.

Approach. Count characters in one string and decrement with the other; all counts must return to zero.

```
bool isAnagram(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    vector<int> cnt(128, 0);
    for (char c : a) cnt[(unsigned char)c]++;
    for (char c : b) if (--cnt[(unsigned char)c] < 0) return false;
    return true;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

42 Group Anagrams Medium

Group words that are anagrams of one another.

Approach. Use a canonical key: either the sorted word or a 26-slot count signature.

```
vector<vector<string>> groupAnagrams(const vector<string>& v) {
    unordered_map<string, vector<string>> m;
    for (const string& w : v) {
        string k = w;
        sort(k.begin(), k.end());
        m[k].push_back(w);
    }
    vector<vector<string>> out;
    for (auto& kv : m) out.push_back(kv.second);
    return out;
}
```

Complexity: $O(n \cdot k \log k)$ time, $O(nk)$ space.

43 Longest Common Prefix Easy

Find the longest common prefix of an array of strings.

Approach. Take the first string as a candidate and shrink it against each subsequent string.

```
string longestCommonPrefix(const vector<string>& v) {
    if (v.empty()) return "";
    string p = v[0];
    for (const string& s : v) {
        size_t i = 0;
        while (i < p.size() && i < s.size() && p[i] == s[i]) i++;
        p = p.substr(0, i);
        if (p.empty()) break;
    }
    return p;
}
```

Complexity: $O(\text{total characters})$ time, $O(1)$ extra space.

44 Reverse Words in a String Medium

Reverse the word order, collapsing extra spaces.

Approach. Reverse the entire string, then reverse each word in place while compacting spaces.

```
string reverseWords(string s) {
    reverse(s.begin(), s.end());
    int w = 0, n = s.size(), i = 0;
    while (i < n) {
        while (i < n && s[i] == ' ') i++;
        if (i == n) break;
        if (w) s[w++] = ' ';
        int start = w;
        while (i < n && s[i] != ' ') s[w++] = s[i++];
        reverse(s.begin() + start, s.begin() + w);
    }
    s.resize(w);
    return s;
}
```

Complexity: $O(n)$ time, $O(1)$ extra space.

45 String to Integer (atoi) Medium

Parse a leading integer from a string with clamping.

Approach. Skip whitespace, read an optional sign, accumulate digits, and clamp on overflow before it happens.

```
int myAtoi(const string& s) {
    int i = 0, n = s.size(), sign = 1;
    long long r = 0;
    while (i < n && s[i] == ' ') i++;
    if (i < n && (s[i] == '+' || s[i] == '-')) sign = (s[i++] == '-') ? -1 : 1;
    while (i < n && isdigit((unsigned char)s[i])) {
        r = r * 10 + (s[i++] - '0');
        if (sign * r > INT_MAX) return INT_MAX;
        if (sign * r < INT_MIN) return INT_MIN;
    }
    return (int)(sign * r);
}
```

Complexity: O(n) time, O(1) space.

46 Valid Palindrome Easy

Check palindromity considering only alphanumerics, case-insensitively.

Approach. Two pointers skipping non-alphanumeric characters.

```
bool isPalindrome(const string& s) {
    int i = 0, j = s.size() - 1;
    while (i < j) {
        while (i < j && !isalnum((unsigned char)s[i])) i++;
        while (i < j && !isalnum((unsigned char)s[j])) j--;
        if (tolower(s[i]) != tolower(s[j])) return false;
        i++; j--;
    }
    return true;
}
```

Complexity: O(n) time, O(1) space.

47 Sliding Window Maximum Hard

Return the maximum of every window of size k.

Approach. Monotonic deque holding indices in decreasing value order; the front is always the window maximum.

```
vector<int> maxSlidingWindow(const vector<int>& a, int k) {
    deque<int> dq;
    vector<int> out;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) out.push_back(a[dq.front()]);
    }
    return out;
}
```

Complexity: O(n) time, O(k) space.

48 Subarray Sum Equals K Medium

Count contiguous subarrays summing to k.

Approach. Prefix sums in a hash map: the number of earlier prefixes equal to sum - k is the count ending here.

```
int subarraySum(const vector<int>& a, int k) {
    unordered_map<int, int> cnt{{0, 1}};
    int sum = 0, res = 0;
    for (int v : a) {
        sum += v;
        auto it = cnt.find(sum - k);
        if (it != cnt.end()) res += it->second;
        cnt[sum]++;
    }
    return res;
}
```

Complexity: O(n) time, O(n) space.

49 Spiral Matrix Medium

Return the elements of a matrix in spiral order.

Approach. Maintain four boundaries and walk them inward, checking for exhaustion between passes.

```
vector<int> spiralOrder(const vector<vector<int>>& m) {
    vector<int> out;
    if (m.empty()) return out;
    int top = 0, bot = m.size() - 1, left = 0, right = m[0].size() - 1;
    while (top <= bot && left <= right) {
        for (int j = left; j <= right; j++) out.push_back(m[top][j]);
        top++;
        for (int i = top; i <= bot; i++) out.push_back(m[i][right]);
        right--;
        if (top <= bot) { for (int j = right; j >= left; j--) out.push_back(m[bot][j]); bot--; }
        if (left <= right) { for (int i = bot; i >= top; i--) out.push_back(m[i][left]); left++; }
    }
    return out;
}
```

Complexity: $O(mn)$ time, $O(1)$ extra space.

50 Set Matrix Zeroes Medium

Zero out the row and column of every zero, in place.

Approach. Use the first row and column as marker storage, handling their own state with two flags.

```
void setZeroes(vector<vector<int>>& m) {
    int R = m.size(), C = m[0].size();
    bool r0 = false, c0 = false;
    for (int j = 0; j < C; j++) if (!m[0][j]) r0 = true;
    for (int i = 0; i < R; i++) if (!m[i][0]) c0 = true;
    for (int i = 1; i < R; i++)
        for (int j = 1; j < C; j++)
            if (!m[i][j]) { m[i][0] = 0; m[0][j] = 0; }
    for (int i = 1; i < R; i++)
        for (int j = 1; j < C; j++)
            if (!m[i][0] || !m[0][j]) m[i][j] = 0;
    if (r0) for (int j = 0; j < C; j++) m[0][j] = 0;
    if (c0) for (int i = 0; i < R; i++) m[i][0] = 0;
}
```

Complexity: $O(mn)$ time, $O(1)$ extra space.

Hashing (Maps)

Every solution in this section is built on a map -- `unordered_map` for $O(1)$ average lookup, or ordered map where key order matters. Sets and fixed count arrays are deliberately avoided so that one structure is practised to fluency. A map is a lookup table: the same object as a tag array or a translation buffer in hardware, with collision handling in software instead of associativity in silicon.

51 Contains Duplicate Easy

Does the array contain any repeated value?

Approach. Count occurrences in a map and stop the moment a key is seen twice.

```
bool containsDuplicate(const vector<int>& a) {
    unordered_map<int,int> cnt;
    for (int v : a)
        if (++cnt[v] > 1) return true;
    return false;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

52 Contains Duplicate II Easy

Is there a repeated value within k indices of itself?

Approach. Map each value to the index where it was last seen; on a repeat compare the index gap.

```
bool containsNearbyDuplicate(const vector<int>& a, int k) {
    unordered_map<int,int> last;
    for (int i = 0; i < (int)a.size(); i++) {
        auto it = last.find(a[i]);
        if (it != last.end() && i - it->second <= k) return true;
        last[a[i]] = i;
    }
    return false;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

53 Majority Element Easy

Find the element appearing more than $n/2$ times.

Approach. Count in a map and return the key whose count crosses $n/2$. Boyer-Moore voting is the $O(1)$ -space alternative worth mentioning.

```
int majorityElement(const vector<int>& a) {
    unordered_map<int,int> cnt;
    int n = a.size();
    for (int v : a)
        if (++cnt[v] > n / 2) return v;
    return -1;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

54 Intersection of Two Arrays Easy

Return the distinct common elements.

Approach. Map every value of the first array to a present flag, then clear the flag as matches are emitted so results stay unique.

```
vector<int> intersection(const vector<int>& a, const vector<int>& b) {
    unordered_map<int,bool> seen;
    for (int v : a) seen[v] = true;
    vector<int> out;
    for (int v : b) {
        auto it = seen.find(v);
        if (it != seen.end() && it->second) {
            out.push_back(v);
            it->second = false;           // emit each value once
        }
    }
    return out;
}
```

Complexity: $O(n+m)$ time, $O(n)$ space.

55 Intersection of Two Arrays II Easy

Return common elements with multiplicity.

Approach. Map value to remaining count; decrement on each match so duplicates are honoured exactly.

```
vector<int> intersectII(const vector<int>& a, const vector<int>& b) {
    unordered_map<int,int> cnt;
    for (int v : a) cnt[v]++;
    vector<int> out;
    for (int v : b) {
        auto it = cnt.find(v);
        if (it != cnt.end() && it->second > 0) { out.push_back(v); it->second--; }
    }
    return out;
}
```

Complexity: $O(n+m)$ time, $O(n)$ space.

56 Happy Number Easy

Repeatedly sum squared digits; does the process reach 1?

Approach. Record every value seen in a map. Reaching 1 succeeds; revisiting a value proves a cycle.

```
int sqSum(int n) {
    int s = 0;
    while (n) { int d = n % 10; s += d * d; n /= 10; }
    return s;
}

bool isHappy(int n) {
    unordered_map<int,bool> seen;
    while (n != 1) {
        if (seen[n]) return false;    // cycle detected
        seen[n] = true;
        n = sqSum(n);
    }
    return true;
}
```

Complexity: $O(\log n)$ time, $O(\log n)$ space.

57 First Unique Character Easy

Index of the first non-repeating character.

Approach. One pass to build a character-to-count map, a second pass to find the first count of one.

```
int firstUniqChar(const string& s) {
    unordered_map<char,int> cnt;
    for (char c : s) cnt[c]++;
    for (int i = 0; i < (int)s.size(); i++)
        if (cnt[s[i]] == 1) return i;
    return -1;
}
```

Complexity: $O(n)$ time, $O(1)$ space (bounded alphabet).

58 Ransom Note Easy

Can the note be built from the magazine's letters?

Approach. Map the magazine's letter counts, then decrement per note letter and fail on a shortfall.

```
bool canConstruct(const string& note, const string& mag) {
    unordered_map<char,int> cnt;
    for (char c : mag) cnt[c]++;
    for (char c : note)
        if (--cnt[c] < 0) return false;
    return true;
}
```

Complexity: $O(n+m)$ time, $O(1)$ space.

59 Isomorphic Strings Easy

Can characters of s be consistently mapped onto t?

Approach. Two maps enforce a bijection: s to t and t back to s. One map alone wrongly accepts many-to-one mappings.

```
bool isIsomorphic(const string& s, const string& t) {
    if (s.size() != t.size()) return false;
    unordered_map<char, char> fwd, rev;
    for (int i = 0; i < (int)s.size(); i++) {
        char a = s[i], b = t[i];
        auto f = fwd.find(a);
        auto r = rev.find(b);
        if (f == fwd.end() && r == rev.end()) { fwd[a] = b; rev[b] = a; }
        else if (f == fwd.end() || r == rev.end() ||
                 f->second != b || r->second != a) return false;
    }
    return true;
}
```

Complexity: O(n) time, O(1) space.

60 Word Pattern Easy

Does a string of words follow a letter pattern?

Approach. The same bijection idea, but mapping pattern letters to whole words and back.

```
bool wordPattern(const string& pattern, const string& str) {
    vector<string> words;
    string w;
    stringstream ss(str);
    while (ss >> w) words.push_back(w);
    if (words.size() != pattern.size()) return false;
    unordered_map<char, string> fwd;
    unordered_map<string, char> rev;
    for (int i = 0; i < (int)words.size(); i++) {
        char c = pattern[i];
        auto f = fwd.find(c);
        auto r = rev.find(words[i]);
        if (f == fwd.end() && r == rev.end()) { fwd[c] = words[i]; rev[words[i]] = c; }
        else if (f == fwd.end() || r == rev.end() ||
                 f->second != words[i] || r->second != c) return false;
    }
    return true;
}
```

Complexity: O(n) time, O(n) space.

61 Longest Consecutive Sequence Medium

Length of the longest run of consecutive integers.

Approach. Map every value to a present flag, then extend a run only from values with no predecessor, so each run is walked exactly once.

```
int longestConsecutive(const vector<int>& a) {
    unordered_map<int, bool> present;
    for (int v : a) present[v] = true;
    int best = 0;
    for (auto& kv : present) {
        int v = kv.first;
        if (present.count(v - 1)) continue; // not the start of a run
        int len = 1, cur = v;
        while (present.count(cur + 1)) { cur++; len++; }
        best = max(best, len);
    }
    return best;
}
```

Complexity: O(n) time, O(n) space.

62 Top K Frequent Elements Medium

Return the k most frequent values.

Approach. Count into a map, then bucket by frequency so no comparison sort is needed.

```
vector<int> topKFrequent(const vector<int>& a, int k) {
    unordered_map<int,int> cnt;
    for (int v : a) cnt[v]++;
    vector<vector<int>> bucket(a.size() + 1);
    for (auto& kv : cnt) bucket[kv.second].push_back(kv.first);
    vector<int> out;
    for (int f = bucket.size() - 1; f >= 1 && (int)out.size() < k; f--)
        for (int v : bucket[f]) {
            out.push_back(v);
            if ((int)out.size() == k) break;
        }
    return out;
}
```

Complexity: O(n) time, O(n) space.

63 Sort Characters By Frequency Medium

Reorder a string so more frequent characters come first.

Approach. Map counts, then sort the map entries by descending count and rebuild the string.

```
string frequencySort(const string& s) {
    unordered_map<char,int> cnt;
    for (char c : s) cnt[c]++;
    vector<pair<char,int>> v(cnt.begin(), cnt.end());
    sort(v.begin(), v.end(),
        [](const pair<char,int>& a, const pair<char,int>& b) {
            return a.second > b.second;
        });
    string out;
    for (auto& kv : v) out.append(kv.second, kv.first);
    return out;
}
```

Complexity: O(n + k log k) time, O(n) space.

64 4Sum II Medium

Count tuples (i,j,k,l) with $A[i] + B[j] + C[k] + D[l] = 0$.

Approach. Meet in the middle: map every pairwise sum of A and B to its frequency, then probe with the negated sums of C and D.

```
int fourSumCount(const vector<int>& A, const vector<int>& B,
                 const vector<int>& C, const vector<int>& D) {
    unordered_map<int,int> ab;
    for (int x : A) for (int y : B) ab[x + y]++;
    int res = 0;
    for (int x : C) for (int y : D) {
        auto it = ab.find(-(x + y));
        if (it != ab.end()) res += it->second;
    }
    return res;
}
```

Complexity: O(n²) time, O(n²) space.

65 Find All Anagrams in a String Medium

Return every start index where p appears as an anagram in s.

Approach. Sliding window of fixed width over a difference map; the window matches when the map holds no non-zero entries.

```
vector<int> findAnagrams(const string& s, const string& p) {
    vector<int> out;
    if (s.size() < p.size()) return out;
    unordered_map<char,int> need;
    for (char c : p) need[c]++;
    int missing = need.size();
    for (int i = 0; i < (int)s.size(); i++) {
        if (--need[s[i]] == 0) missing--;
        if (i >= (int)p.size()) {
            int j = i - p.size();
            if (++need[s[j]] == 1) missing++;
        }
        if (missing == 0 && i + 1 >= (int)p.size())
            out.push_back(i - p.size() + 1);
    }
    return out;
}
```

Complexity: O(n) time, O(1) space.

66 Longest Substring with At Most K Distinct Characters Medium

Longest substring containing no more than k distinct characters.

Approach. Sliding window with a character-to-count map; shrink from the left while the map holds more than k keys.

```
int lengthOfLongestSubstringKDistinct(const string& s, int k) {
    unordered_map<char,int> cnt;
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        cnt[s[r]]++;
        while ((int)cnt.size() > k) {
            if (--cnt[s[l]] == 0) cnt.erase(s[l]);
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

Complexity: O(n) time, O(k) space.

67 Valid Sudoku Medium

Verify that a filled Sudoku board breaks no rule.

Approach. Encode each constraint as a distinct string key -- row, column, and box -- and count occurrences in one map.

```
bool isValidSudoku(const vector<vector<char>>& b) {
    unordered_map<string,int> seen;
    for (int i = 0; i < 9; i++)
        for (int j = 0; j < 9; j++) {
            char c = b[i][j];
            if (c == '.') continue;
            string r = "r" + to_string(i) + "#" + c;
            string col = "c" + to_string(j) + "#" + c;
            string box = "b" + to_string(i/3) + to_string(j/3) + "#" + c;
            if (++seen[r] > 1 || ++seen[col] > 1 || ++seen[box] > 1) return false;
        }
    return true;
}
```

Complexity: O(81) time, O(81) space.

68 Group Shifted Strings Medium

Group strings that are cyclic shifts of one another.

Approach. Normalise each word by shifting so it starts at 'a'; that canonical form is the map key.

```
vector<vector<string>> groupStrings(const vector<string>& v) {
    unordered_map<string, vector<string>> m;
    for (const string& w : v) {
        string key;
        int shift = w[0] - 'a';
        for (char c : w) key += (char)('a' + ((c - 'a' - shift + 26) % 26));
        m[key].push_back(w);
    }
    vector<vector<string>> out;
    for (auto& kv : m) out.push_back(kv.second);
    return out;
}
```

Complexity: O(total characters) time and space.

69 Subdomain Visit Count Medium

Aggregate visit counts across every parent domain.

Approach. Map full domain strings to counts, adding each suffix of the domain as its own key.

```
vector<string> subdomainVisits(const vector<string>& cpdomains) {
    unordered_map<string,int> cnt;
    for (const string& s : cpdomains) {
        int sp = s.find(' ');
        int num = stoi(s.substr(0, sp));
        string dom = s.substr(sp + 1);
        cnt[dom] += num;
        for (int i = 0; i < (int)dom.size(); i++)
            if (dom[i] == '.') cnt[dom.substr(i + 1)] += num;
    }
    vector<string> out;
    for (auto& kv : cnt) out.push_back(to_string(kv.second) + " " + kv.first);
    return out;
}
```

Complexity: O(total characters) time and space.

70 Insert Delete GetRandom in O(1) Medium

Support insert, remove, and uniform random access, all in constant time.

Approach. A map from value to its index in a vector. Removal swaps the target with the last element so the vector stays dense.

```
class RandomizedSet {
    unordered_map<int,int> pos;    // value -> index in vec
    vector<int> vec;
public:
    bool insert(int v) {
        if (pos.count(v)) return false;
        pos[v] = vec.size();
        vec.push_back(v);
        return true;
    }
    bool remove(int v) {
        auto it = pos.find(v);
        if (it == pos.end()) return false;
        int idx = it->second, last = vec.back();
        vec[idx] = last;
        pos[last] = idx;    // repair the moved element
        vec.pop_back();
        pos.erase(it);
        return true;
    }
    int getRandom() { return vec[rand() % vec.size()]; }
};
```

Complexity: O(1) average per operation, O(n) space.

Hardware connection. The swap-with-last trick keeps storage contiguous, the same motivation as compacting a free list in an allocator.

71 Time Based Key-Value Store Medium

Store values with timestamps and retrieve the newest value at or before a query time.

Approach. An ordered map from key to a time-sorted vector; binary search the vector on lookup.

```
class TimeMap {
    unordered_map<string, vector<pair<int,string>>> m;
public:
    void set(const string& k, const string& v, int t) {
        m[k].push_back({t, v});    // timestamps arrive increasing
    }
    string get(const string& k, int t) {
        auto it = m.find(k);
        if (it == m.end()) return "";
        auto& vec = it->second;
        int lo = 0, hi = vec.size() - 1;
        string res;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (vec[mid].first <= t) { res = vec[mid].second; lo = mid + 1; }
            else hi = mid - 1;
        }
        return res;
    }
};
```

Complexity: O(1) set, O(log n) get, O(n) space.

72 Bulls and Cows Medium

Report exact matches (bulls) and misplaced matches (cows).

Approach. Bulls are counted positionally in one pass; a digit-to-count map resolves the cows from the leftovers.

```
string getHint(const string& secret, const string& guess) {
    unordered_map<char,int> sCnt, gCnt;
    int bulls = 0, cows = 0;
    for (int i = 0; i < (int)secret.size(); i++) {
        if (secret[i] == guess[i]) bulls++;
        else { sCnt[secret[i]]++; gCnt[guess[i]]++; }
    }
    for (auto& kv : gCnt) {
        auto it = sCnt.find(kv.first);
        if (it != sCnt.end()) cows += min(kv.second, it->second);
    }
    return to_string(bulls) + "A" + to_string(cows) + "B";
}
```

Complexity: O(n) time, O(1) space.

73 Longest Palindrome Buildable Easy

Longest palindrome length constructible from a multiset of letters.

Approach. Map letter counts; every pair contributes two, and one odd leftover may sit in the centre.

```
int longestPalindrome(const string& s) {
    unordered_map<char,int> cnt;
    for (char c : s) cnt[c]++;
    int len = 0;
    bool odd = false;
    for (auto& kv : cnt) {
        len += (kv.second / 2) * 2;
        if (kv.second % 2) odd = true;
    }
    return len + (odd ? 1 : 0);
}
```

Complexity: $O(n)$ time, $O(1)$ space.

74 Encode and Decode TinyURL Medium

Design a URL shortener.

Approach. Two maps giving a bijection between the long URL and a generated short code, so repeated encodes are stable.

```
class Codec {
    unordered_map<string,string> longToShort, shortToLong;
    int counter = 0;
public:
    string encode(const string& longUrl) {
        auto it = longToShort.find(longUrl);
        if (it != longToShort.end()) return it->second;
        string code = "http://tinyurl.com/" + to_string(counter++);
        longToShort[longUrl] = code;
        shortToLong[code] = longUrl;
        return code;
    }
    string decode(const string& shortUrl) {
        auto it = shortToLong.find(shortUrl);
        return it == shortToLong.end() ? "" : it->second;
    }
};
```

Complexity: $O(1)$ average per operation, $O(n)$ space.

Binary Search and Sorting

Interviewers use binary search to test precision. Know the invariant you are maintaining and be able to state why your loop terminates. Binary search on the answer space is the variant most candidates miss.

75 Binary Search Easy

Find a target in a sorted array.

Approach. Half-open invariant with mid computed as $lo + (hi-lo)/2$ to avoid overflow.

```
int binarySearch(const vector<int>& a, int t) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
```

Complexity: $O(\log n)$ time, $O(1)$ space.

76 First and Last Occurrence Medium

Find the first and last index of a target.

Approach. Two biased binary searches: on equality keep searching left for the first, right for the last.

```
int bound(const vector<int>& a, int t, bool first) {
    int lo = 0, hi = a.size() - 1, res = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) { res = mid; first ? hi = mid - 1 : lo = mid + 1; }
        else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return res;
}
```

Complexity: $O(\log n)$ time, $O(1)$ space.

77 Search in Rotated Sorted Array Medium

Search a sorted array rotated at an unknown pivot.

Approach. One half is always sorted; test which, then decide whether the target lies inside it.

```
int searchRotated(const vector<int>& a, int t) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[lo] <= a[mid]) {
            if (a[lo] <= t && t < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {
            if (a[mid] < t && t <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
```

Complexity: $O(\log n)$ time, $O(1)$ space.

78 Minimum in Rotated Sorted Array Medium

Find the smallest element of a rotated sorted array.

Approach. Compare mid against the right end to decide which side holds the rotation point.

```
int findMin(const vector<int>& a) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[hi]) lo = mid + 1; else hi = mid;
    }
    return a[lo];
}
```

Complexity: $O(\log n)$ time, $O(1)$ space.

79 Search a 2D Matrix Medium

Search a matrix with sorted rows where each row starts after the previous ends.

Approach. Treat it as a flat sorted array and map the index back with divide and modulo.

```
bool searchMatrix(const vector<vector<int>>& m, int t) {
    int R = m.size(), C = m[0].size();
    int lo = 0, hi = R * C - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int v = m[mid / C][mid % C];
        if (v == t) return true;
        if (v < t) lo = mid + 1; else hi = mid - 1;
    }
    return false;
}
```

Complexity: $O(\log(mn))$ time, $O(1)$ space.

80 Find Peak Element Medium

Return the index of any element greater than its neighbours.

Approach. Binary search on the slope: move toward the rising side, which must contain a peak.

```
int findPeakElement(const vector<int>& a) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < a[mid + 1]) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

Complexity: $O(\log n)$ time, $O(1)$ space.

81 Integer Square Root Easy

Compute $\text{floor}(\sqrt{x})$ without floating point.

Approach. Binary search the answer space; compare with 64-bit products to avoid overflow.

```
int mySqrt(int x) {
    long long lo = 0, hi = x, res = 0;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (mid * mid <= x) { res = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return (int)res;
}
```

Complexity: $O(\log x)$ time, $O(1)$ space.

Hardware connection. The restoring-division style shift-and-subtract square root is the hardware analogue -- worth mentioning.

82 Koko Eating Bananas Medium

Find the minimum eating rate to finish within h hours.

Approach. Binary search on the answer: feasibility is monotonic in the rate.

```
bool feasible(const vector<int>& p, int h, int k) {
    long long hrs = 0;
    for (int v : p) hrs += (v + k - 1) / k;
    return hrs <= h;
}

int minEatingSpeed(const vector<int>& p, int h) {
    int lo = 1, hi = *max_element(p.begin(), p.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (feasible(p, h, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

Complexity: $O(n \log \max P)$ time, $O(1)$ space.

83 Capacity to Ship Packages in D Days Medium

Least capacity that ships all packages within D days.

Approach. Binary search on capacity between $\max(\text{weight})$ and $\sum(\text{weights})$.

```
int shipWithinDays(const vector<int>& w, int D) {
    int lo = *max_element(w.begin(), w.end());
    int hi = accumulate(w.begin(), w.end(), 0);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2, days = 1, cur = 0;
        for (int x : w) {
            if (cur + x > mid) { days++; cur = 0; }
            cur += x;
        }
        if (days <= D) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

Complexity: $O(n \log \text{sum})$ time, $O(1)$ space.

84 Median of Two Sorted Arrays Hard

Find the median of two sorted arrays in logarithmic time.

Approach. Binary search the partition point in the smaller array so that left and right halves balance.

```
double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
    if (A.size() > B.size()) return findMedianSortedArrays(B, A);
    int m = A.size(), n = B.size(), half = (m + n + 1) / 2;
    int lo = 0, hi = m;
    while (lo <= hi) {
        int i = (lo + hi) / 2, j = half - i;
        int Al = i > 0 ? A[i-1] : INT_MIN, Ar = i < m ? A[i] : INT_MAX;
        int Bl = j > 0 ? B[j-1] : INT_MIN, Br = j < n ? B[j] : INT_MAX;
        if (Al <= Br && Bl <= Ar) {
            if ((m + n) % 2) return max(Al, Bl);
            return (max(Al, Bl) + min(Ar, Br)) / 2.0;
        }
        if (Al > Br) hi = i - 1; else lo = i + 1;
    }
    return 0.0;
}
```

Complexity: $O(\log \min(m,n))$ time, $O(1)$ space.

85 Merge Sort Medium

Implement merge sort.

Approach. Divide, recurse, and merge with a scratch buffer. Stable, and the basis for counting inversions.

```
void mergeSort(vector<int>& a, int l, int r, vector<int>& tmp) {
    if (r - l <= 1) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, l, mid, tmp);
    mergeSort(a, mid, r, tmp);
    int i = l, j = mid, k = l;
    while (i < mid && j < r) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
    for (int t = l; t < r; t++) a[t] = tmp[t];
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

Hardware connection. The merge network generalises to a bitonic sorter, which is the standard hardware sort.

86 Quickselect: Kth Largest Element Medium

Find the kth largest element without full sorting.

Approach. Partition around a pivot and recurse into the side containing the target rank.

```
int quickselect(vector<int>& a, int k) { // k is 1-indexed from largest
    int target = a.size() - k, lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int p = a[hi], w = lo;
        for (int i = lo; i < hi; i++) if (a[i] < p) swap(a[i], a[w++]);
        swap(a[w], a[hi]);
        if (w == target) return a[w];
        if (w < target) lo = w + 1; else hi = w - 1;
    }
    return -1;
}
```

Complexity: $O(n)$ average, $O(n^2)$ worst; $O(1)$ space.

87 Count Inversions Hard

Count pairs $i < j$ with $a[i] > a[j]$.

Approach. Piggyback on merge sort: when taking from the right half, every remaining left element forms an inversion.

```
long long countInv(vector<int>& a, int l, int r, vector<int>& tmp) {
    if (r - l <= 1) return 0;
    int mid = l + (r - l) / 2;
    long long inv = countInv(a, l, mid, tmp) + countInv(a, mid, r, tmp);
    int i = l, j = mid, k = l;
    while (i < mid && j < r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { inv += mid - i; tmp[k++] = a[j++]; }
    }
    while (i < mid) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
    for (int t = l; t < r; t++) a[t] = tmp[t];
    return inv;
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

Stacks, Queues, and Monotonic Structures

Includes the ring-buffer FIFO, which is the single most hardware-relevant data structure on this list. You have built one in RTL; being able to write and reason about it in C is a question you should welcome.

88 Valid Parentheses Easy

Check that brackets are correctly matched and nested.

Approach. Push openers, pop and compare on closers, and require an empty stack at the end.

```
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char o = st.top(); st.pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') ||
                (c == '}' && o != '{')) return false;
        }
    }
    return st.empty();
}
```

Complexity: $O(n)$ time, $O(n)$ space.

89 Min Stack Medium

Stack supporting push, pop, top, and getMin in $O(1)$.

Approach. Store the running minimum alongside each element, so popping restores the previous minimum for free.

```
class MinStack {
    stack<pair<int,int>> st;    // (value, min so far)
public:
    void push(int x) {
        int m = st.empty() ? x : min(x, st.top().second);
        st.push({x, m});
    }
    void pop() { st.pop(); }
    int top() { return st.top().first; }
    int getMin() { return st.top().second; }
};
```

Complexity: $O(1)$ per operation, $O(n)$ space.

90 Queue Using Two Stacks Easy

Implement a FIFO queue with two LIFO stacks.

Approach. Push onto the input stack; on pop, if the output stack is empty, drain input into it. Amortised $O(1)$.

```
class MyQueue {
    stack<int> in, out;
    void shift() { while (!in.empty()) { out.push(in.top()); in.pop(); } }
public:
    void push(int x) { in.push(x); }
    int pop() { if (out.empty()) shift(); int v = out.top(); out.pop(); return v; }
    int peek() { if (out.empty()) shift(); return out.top(); }
    bool empty() { return in.empty() && out.empty(); }
};
```

Complexity: Amortised $O(1)$ per operation, $O(n)$ space.

91 Next Greater Element Medium

For each element find the next strictly greater element to its right.

Approach. Monotonic decreasing stack of indices; on a larger value, pop and resolve those indices.

```
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> out(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) { out[st.top()] = a[i]; st.pop(); }
        st.push(i);
    }
    return out;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

92 Daily Temperatures Medium

Days to wait for a warmer temperature.

Approach. Identical monotonic stack, but record index distance rather than value.

```
vector<int> dailyTemperatures(const vector<int>& t) {
    int n = t.size();
    vector<int> out(n, 0);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && t[st.top()] < t[i]) { out[st.top()] = i - st.top(); st.pop(); }
        st.push(i);
    }
    return out;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

93 Largest Rectangle in Histogram Hard

Largest rectangle fitting under a histogram.

Approach. Monotonic increasing stack; when a shorter bar arrives, pop and compute areas with the popped bar as the height.

```
int largestRectangleArea(vector<int> h) {
    h.push_back(0);
    stack<int> st;
    int best = 0;
    for (int i = 0; i < (int)h.size(); i++) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int ht = h[st.top()]; st.pop();
            int left = st.empty() ? -1 : st.top();
            best = max(best, ht * (i - left - 1));
        }
        st.push(i);
    }
    return best;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

94 Evaluate Reverse Polish Notation Medium

Evaluate an RPN expression.

Approach. Push operands; on an operator pop two, apply, push back. Mind operand order for minus and divide.

```
int evalRPN(const vector<string>& tok) {
    stack<long long> st;
    for (const string& t : tok) {
        if (t.size() > 1 || isdigit((unsigned char)t[0])) { st.push(stoll(t)); continue; }
        long long b = st.top(); st.pop();
        long long a = st.top(); st.pop();
        if (t == "+") st.push(a + b);
        else if (t == "-") st.push(a - b);
        else if (t == "*") st.push(a * b);
        else st.push(a / b);
    }
    return (int)st.top();
}
```

Complexity: $O(n)$ time, $O(n)$ space.

95 Ring Buffer / Circular FIFO Medium

Implement a fixed-capacity FIFO over a circular array with full and empty detection.

Approach. Keep read and write pointers with one extra wrap bit. Equal pointers with equal wrap bits means empty; equal pointers with differing wrap bits means full. This avoids sacrificing a slot.

```
template <typename T, int DEPTH>
class RingFifo {
    T buf[DEPTH];
    unsigned rptr = 0, wptr = 0; // low bits index, top bit is the wrap flag
    static const unsigned MASK = DEPTH - 1; // DEPTH must be a power of two
    static const unsigned WRAP = DEPTH;
public:
    bool empty() const { return rptr == wptr; }
    bool full() const { return (rptr & MASK) == (wptr & MASK) &&
        (rptr & WRAP) != (wptr & WRAP); }
    int count() const { return (wptr - rptr) & (2 * DEPTH - 1); }
    bool push(const T& v) {
        if (full()) return false;
        buf[wptr & MASK] = v;
        wptr = (wptr + 1) & (2 * DEPTH - 1);
        return true;
    }
    bool pop(T& v) {
        if (empty()) return false;
        v = buf[rptr & MASK];
        rptr = (rptr + 1) & (2 * DEPTH - 1);
        return true;
    }
};
```

Complexity: O(1) per operation, O(DEPTH) space.

Hardware connection. This is the software twin of your synchronous FIFO. The extra wrap bit is precisely the trick used in RTL, and in an asynchronous FIFO those pointers are Gray-coded before crossing clock domains. Expect a follow-up on why the pointers are one bit wider than the address.

96 Design Circular Queue Medium

Implement a circular queue with a fixed capacity.

Approach. Array plus head index and size counter; size disambiguates full from empty without a wrap bit.

```
class MyCircularQueue {
    vector<int> b; int head = 0, sz = 0, cap;
public:
    MyCircularQueue(int k) : b(k), cap(k) {}
    bool enqueue(int v) {
        if (sz == cap) return false;
        b[(head + sz) % cap] = v; sz++;
        return true;
    }
    bool dequeue() {
        if (!sz) return false;
        head = (head + 1) % cap; sz--;
        return true;
    }
    int Front() { return sz ? b[head] : -1; }
    int Rear() { return sz ? b[(head + sz - 1) % cap] : -1; }
    bool isEmpty() const { return sz == 0; }
    bool isFull() const { return sz == cap; }
};
```

Complexity: O(1) per operation, O(k) space.

97 Asteroid Collision Medium

Simulate collisions between right- and left-moving asteroids.

Approach. Stack of surviving asteroids; a left-mover collides with the stack top while that top moves right.

```
vector<int> asteroidCollision(const vector<int>& a) {
    vector<int> st;
    for (int x : a) {
        bool alive = true;
        while (alive && x < 0 && !st.empty() && st.back() > 0) {
            if (st.back() < -x) { st.pop_back(); continue; }
            if (st.back() == -x) st.pop_back();
            alive = false;
        }
        if (alive) st.push_back(x);
    }
    return st;
}
```

Complexity: O(n) time, O(n) space.

98 Decode String Medium

Expand encoded strings such as 3[a2[c]] into accccc.

Approach. Two stacks, one for repeat counts and one for partial strings, unwound on the closing bracket.

```
string decodeString(const string& s) {
    stack<int> nums;
    stack<string> strs;
    string cur;
    int num = 0;
    for (char c : s) {
        if (isdigit((unsigned char)c)) num = num * 10 + (c - '0');
        else if (c == '[') { nums.push(num); strs.push(cur); num = 0; cur.clear(); }
        else if (c == ']') {
            string tmp = cur;
            for (int i = 1; i < nums.top(); i++) cur += tmp;
            cur = strs.top() + cur;
            nums.pop(); strs.pop();
        } else cur += c;
    }
    return cur;
}
```

Complexity: O(output length) time and space.

99 Remove K Digits Medium

Remove k digits to leave the smallest possible number.

Approach. Greedy monotonic stack: pop larger preceding digits while removals remain.

```
string removeKdigits(const string& num, int k) {
    string st;
    for (char c : num) {
        while (k && !st.empty() && st.back() > c) { st.pop_back(); k--; }
        st.push_back(c);
    }
    st.resize(st.size() - k);
    int i = 0;
    while (i < (int)st.size() && st[i] == '0') i++;
    string r = st.substr(i);
    return r.empty() ? "0" : r;
}
```

Complexity: O(n) time, O(n) space.

100 Simplify Unix Path Medium

Canonicalise an absolute Unix path.

Approach. Split on slashes and use a stack: '..' pops, '.' and empty tokens are skipped.

```
string simplifyPath(const string& p) {
    vector<string> st;
    string tok;
    stringstream ss(p);
    while (getline(ss, tok, '/')) {
        if (tok.empty() || tok == ".") continue;
        if (tok == "..") { if (!st.empty()) st.pop_back(); }
        else st.push_back(tok);
    }
    string out;
    for (const string& s : st) out += "/" + s;
    return out.empty() ? "/" : out;
}
```

Complexity: O(n) time, O(n) space.

Linked Lists

Pointer discipline. TI and Nvidia use these less for algorithmic insight than to see whether you can manipulate pointers without leaking, losing the list, or dereferencing null. Two habits solve most of them: a dummy head node to remove special cases at the front, and drawing the pointers before writing a line of code.

Shared definitions used in this section:

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};
```

101 Reverse a Linked List Easy

Reverse a singly linked list, iteratively and recursively.

Approach. Iterative: walk with three pointers, re-pointing each next backwards. Save the successor before overwriting it.

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nxt = head->next; // save before overwrite
        head->next = prev;
        prev = head;
        head = nxt;
    }
    return prev;
}

ListNode* reverseRec(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* rest = reverseRec(head->next);
    head->next->next = head;
    head->next = nullptr;
    return rest;
}
```

Complexity: $O(n)$ time; $O(1)$ iterative, $O(n)$ recursive stack.

102 Reverse a Sublist (Positions m to n) Medium

Reverse only the nodes between positions m and n.

Approach. Walk to the node before m, then repeatedly splice the following node to the front of the sublist.

```
ListNode* reverseBetween(ListNode* head, int m, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    for (int i = 1; i < m; i++) prev = prev->next;
    ListNode* cur = prev->next;
    for (int i = m; i < n; i++) {
        ListNode* mv = cur->next;
        cur->next = mv->next;
        mv->next = prev->next;
        prev->next = mv;
    }
    return dummy.next;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

103 Reverse Nodes in k-Group Hard

Reverse every consecutive group of k nodes.

Approach. Check that k nodes remain, reverse that block, then reconnect and advance to the next group.

```
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* groupPrev = &dummy;
    while (true) {
        ListNode* probe = groupPrev;
        for (int i = 0; i < k && probe; i++) probe = probe->next;
        if (!probe) break; // fewer than k remain
        ListNode* prev = probe->next;
        ListNode* cur = groupPrev->next;
        ListNode* first = cur;
        for (int i = 0; i < k; i++) {
            ListNode* nxt = cur->next;
            cur->next = prev;
            prev = cur;
            cur = nxt;
        }
        groupPrev->next = prev;
        groupPrev = first;
    }
    return dummy.next;
}
```

Complexity: O(n) time, O(1) space.

104 Detect a Cycle Easy

Determine whether a linked list contains a cycle.

Approach. Floyd's tortoise and hare: a fast pointer laps a slow one if and only if a cycle exists.

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Complexity: O(n) time, O(1) space.

105 Find Cycle Start Medium

Return the node where the cycle begins.

Approach. After the meeting point, reset one pointer to the head and advance both one step at a time; they meet at the entry.

```
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}
```

Complexity: O(n) time, O(1) space.

106 Cycle Length Medium

Return the number of nodes in the cycle.

Approach. Once the pointers meet, hold one still and walk the other until it returns.

```
int cycleLength(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            int len = 1;
            for (ListNode* p = slow->next; p != slow; p = p->next) len++;
            return len;
        }
    }
    return 0;
}
```

Complexity: O(n) time, O(1) space.

107 Middle of the List Easy

Return the middle node (second middle if the length is even).

Approach. Fast pointer moves twice per slow step; when fast runs off the end, slow is at the middle.

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
```

Complexity: O(n) time, O(1) space.

108 Merge Two Sorted Lists Easy

Merge two sorted lists into one sorted list.

Approach. Dummy head plus a tail pointer removes every special case; attach the remaining list at the end.

```
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0), *tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
```

Complexity: O(n+m) time, O(1) space.

109 Merge K Sorted Lists Hard

Merge k sorted linked lists into one.

Approach. Min-heap holding the current head of each list; pop the smallest and push its successor.

```
ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
    for (ListNode* l : lists) if (l) pq.push(l);
    ListNode dummy(0), *tail = &dummy;
    while (!pq.empty()) {
        ListNode* n = pq.top(); pq.pop();
        tail->next = n; tail = n;
        if (n->next) pq.push(n->next);
    }
    return dummy.next;
}
```

Complexity: O(N log k) time, O(k) space.

110 Sort a Linked List Medium

Sort a linked list in O(n log n) time.

Approach. Merge sort: split at the middle with slow/fast pointers, sort each half, then merge. Quicksort is a poor fit without random access.

```
ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode *slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* mid = slow->next;
    slow->next = nullptr; // cut
    return mergeTwoLists(sortList(head), sortList(mid));
}
```

Complexity: O(n log n) time, O(log n) recursion depth.

111 Remove Nth Node From End Medium

Delete the nth node counting from the tail.

Approach. Advance a lead pointer n steps, then move both until the lead reaches the last node.

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode *fast = &dummy, *slow = &dummy;
    while (n--) fast = fast->next;
    while (fast->next) { fast = fast->next; slow = slow->next; }
    slow->next = slow->next->next;
    return dummy.next;
}
```

Complexity: O(n) time, O(1) space.

112 Remove Duplicates from Sorted List Easy

Delete duplicate values, keeping one of each.

Approach. Single pass, skipping nodes equal to the current one.

```
ListNode* deleteDuplicates(ListNode* head) {
    ListNode* cur = head;
    while (cur && cur->next) {
        if (cur->val == cur->next->val) cur->next = cur->next->next;
        else cur = cur->next;
    }
    return head;
}
```

Complexity: O(n) time, O(1) space.

113 Remove Duplicates from Sorted List II Medium

Delete every node that has a duplicate, keeping none of them.

Approach. Dummy head plus a trailing pointer; when a run of equal values is found, skip the entire run.

```
ListNode* deleteDuplicatesII(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    ListNode* cur = head;
    while (cur) {
        if (cur->next && cur->val == cur->next->val) {
            int dup = cur->val;
            while (cur && cur->val == dup) cur = cur->next;
            prev->next = cur;
            // drop the whole run
        } else {
            prev = cur;
            cur = cur->next;
        }
    }
    return dummy.next;
}
```

Complexity: O(n) time, O(1) space.

114 Remove Elements by Value Easy

Delete every node holding a given value.

Approach. Dummy head so removing the first node needs no special case.

```
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* cur = &dummy;
    while (cur->next) {
        if (cur->next->val == val) cur->next = cur->next->next;
        else cur = cur->next;
    }
    return dummy.next;
}
```

Complexity: O(n) time, O(1) space.

115 Delete a Node Given Only That Node Easy

Delete a node from the middle of a list when only a pointer to it is given.

Approach. The predecessor is unreachable, so copy the successor's value into this node and unlink the successor instead.

```
void deleteNode(ListNode* node) {
    node->val = node->next->val;
    ListNode* dead = node->next;
    node->next = dead->next;
    delete dead;
    // fails only if node is the tail
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. A classic trick question. The right follow-up answer is that it cannot work for the tail, which is exactly why hardware linked structures usually keep a doubly linked list or a tail sentinel.

116 Partition List Medium

Reorder so all nodes below x precede those greater or equal, preserving order.

Approach. Build two separate lists with their own dummy heads, then concatenate.

```
ListNode* partition(ListNode* head, int x) {
    ListNode lessDummy(0), geDummy(0);
    ListNode *lt = &lessDummy, *ge = &geDummy;
    while (head) {
        if (head->val < x) { lt->next = head; lt = lt->next; }
        else { ge->next = head; ge = ge->next; }
        head = head->next;
    }
    ge->next = nullptr;           // terminate
    lt->next = geDummy.next;
    return lessDummy.next;
}
```

Complexity: O(n) time, O(1) space.

117 Odd Even Linked List Medium

Group nodes at odd positions before those at even positions.

Approach. Weave two chains in one pass and join the odd tail to the even head.

```
ListNode* oddEvenList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode *odd = head, *even = head->next, *evenHead = even;
    while (even && even->next) {
        odd->next = even->next; odd = odd->next;
        even->next = odd->next; even = even->next;
    }
    odd->next = evenHead;
    return head;
}
```

Complexity: O(n) time, O(1) space.

118 Rotate List Medium

Rotate the list to the right by k places.

Approach. Close the list into a ring, compute the new tail at length minus k modulo length, then break the ring there.

```
ListNode* rotateRight(ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;
    int n = 1;
    ListNode* tail = head;
    while (tail->next) { tail = tail->next; n++; }
    tail->next = head;           // make it circular
    k %= n;
    int steps = n - k;
    ListNode* newTail = tail;
    for (int i = 0; i < steps; i++) newTail = newTail->next;
    head = newTail->next;
    newTail->next = nullptr;     // break the ring
    return head;
}
```

Complexity: O(n) time, O(1) space.

119 Palindrome Linked List Medium

Check whether a list reads the same forwards and backwards.

Approach. Find the middle, reverse the second half, compare, then restore the list before returning.

```
bool isPalindrome(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = reverseList(slow);
    ListNode *p = head, *q = second;
    bool ok = true;
    while (q) {
        if (p->val != q->val) { ok = false; break; }
        p = p->next; q = q->next;
    }
    reverseList(second);           // restore
    return ok;
}
```

Complexity: O(n) time, O(1) space.

120 Intersection of Two Linked Lists Easy

Find the node where two lists merge.

Approach. Two pointers that switch to the other list on reaching the end travel equal total distance and meet at the junction.

```
ListNode* getIntersectionNode(ListNode* a, ListNode* b) {
    ListNode *p = a, *q = b;
    while (p != q) {
        p = p ? p->next : b;
        q = q ? q->next : a;
    }
    return p; // nullptr if they never intersect
}
```

Complexity: $O(n+m)$ time, $O(1)$ space.

121 Add Two Numbers Medium

Add two numbers stored as reversed digit lists.

Approach. Walk both lists with an explicit carry, continuing while either list or the carry remains.

```
ListNode* addTwoNumbers(ListNode* a, ListNode* b) {
    ListNode dummy(0), *tail = &dummy;
    int carry = 0;
    while (a || b || carry) {
        int s = carry;
        if (a) { s += a->val; a = a->next; }
        if (b) { s += b->val; b = b->next; }
        carry = s / 10;
        tail->next = new ListNode(s % 10);
        tail = tail->next;
    }
    return dummy.next;
}
```

Complexity: $O(\max(n,m))$ time, $O(1)$ extra space.

Hardware connection. The same carry-propagation structure as the bitwise adder in the first section, just in base 10.

122 Reorder List Medium

Reorder L_0 to L_n , then L_1 to L_{n-1} , and so on.

Approach. Split at the middle, reverse the second half, then interleave the two halves.

```
void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    ListNode *slow = head, *fast = head;
    while (fast->next && fast->next->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = reverseList(slow->next);
    slow->next = nullptr;
    ListNode* first = head;
    while (second) {
        ListNode *f = first->next, *s = second->next;
        first->next = second;
        second->next = f;
        first = f; second = s;
    }
}
```

Complexity: $O(n)$ time, $O(1)$ space.

123 Swap Nodes in Pairs Medium

Swap every two adjacent nodes.

Approach. Dummy head, then relink three pointers per pair. Swapping nodes rather than values is the point.

```
ListNode* swapPairs(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    while (prev->next && prev->next->next) {
        ListNode *a = prev->next, *b = a->next;
        a->next = b->next;
        b->next = a;
        prev->next = b;
        prev = a;
    }
    return dummy.next;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

124 Split a List into k Parts Medium

Divide a list into k consecutive parts differing in size by at most one.

Approach. Measure the length, then give the first (length mod k) parts one extra node.

```
vector<ListNode*> splitListToParts(ListNode* head, int k) {
    int n = 0;
    for (ListNode* p = head; p; p = p->next) n++;
    int size = n / k, extra = n % k;
    vector<ListNode*> out(k, nullptr);
    ListNode* cur = head;
    for (int i = 0; i < k && cur; i++) {
        out[i] = cur;
        int len = size + (i < extra ? 1 : 0);
        for (int j = 1; j < len; j++) cur = cur->next;
        ListNode* nxt = cur->next;
        cur->next = nullptr;      // terminate this part
        cur = nxt;
    }
    return out;
}
```

Complexity: $O(n + k)$ time, $O(k)$ space.

125 Copy List with Random Pointer Medium

Deep-copy a list where each node also has a pointer to an arbitrary node.

Approach. Map original nodes to clones in a first pass, then wire next and random pointers in a second pass using that map.

```
struct RNode {
    int val;
    RNode *next, *random;
    RNode(int v) : val(v), next(nullptr), random(nullptr) {}
};

RNode* copyRandomList(RNode* head) {
    if (!head) return nullptr;
    unordered_map<RNode*, RNode*> clone;
    for (RNode* p = head; p; p = p->next)
        clone[p] = new RNode(p->val);
    for (RNode* p = head; p; p = p->next) {
        clone[p]->next = p->next ? clone[p->next] : nullptr;
        clone[p]->random = p->random ? clone[p->random] : nullptr;
    }
    return clone[head];
}
```

Complexity: $O(n)$ time, $O(n)$ space.

Hardware connection. The $O(1)$ -space variant interleaves clones into the original list, then unweaves them. Worth knowing as the follow-up.

126 Flatten a Multilevel Doubly Linked List Medium

Flatten a doubly linked list whose nodes may have a child list.

Approach. Depth-first: on meeting a child, splice it in between the node and its successor, remembering the successor on a stack.

```
struct DNode {
    int val;
    DNode *prev, *next, *child;
    DNode(int v) : val(v), prev(nullptr), next(nullptr), child(nullptr) {}
};

DNode* flatten(DNode* head) {
    for (DNode* cur = head; cur; cur = cur->next) {
        if (!cur->child) continue;
        DNode* nxt = cur->next;
        DNode* ch = cur->child;
        cur->next = ch;
        ch->prev = cur;
        cur->child = nullptr;
        DNode* tail = ch;
        while (tail->next) tail = tail->next;
        tail->next = nxt;
        if (nxt) nxt->prev = tail;
    }
    return head;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

127 Doubly Linked List Insert and Delete Easy

Implement insertion after a node and deletion of a node in a doubly linked list.

Approach. Four pointer updates for an insert, two for a delete. Sentinel head and tail nodes remove every null check.

```
struct DLNode {
    int val;
    DLNode *prev, *next;
    DLNode(int v) : val(v), prev(nullptr), next(nullptr) {}
};

void insertAfter(DLNode* node, DLNode* fresh) {
    fresh->next = node->next;
    fresh->prev = node;
    if (node->next) node->next->prev = fresh;
    node->next = fresh;
}

void unlink(DLNode* node) {
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
    node->prev = node->next = nullptr;
}
```

Complexity: $O(1)$ per operation, $O(1)$ space.

Hardware connection. This is the list underneath the LRU cache in the hardware crossover section -- constant-time unlink is exactly why an LRU needs a doubly linked list rather than a singly linked one.

Trees and Binary Search Trees

Recursion discipline and traversal fluency. Know the iterative forms too -- being asked to do inorder traversal without recursion is common.

Shared definitions used in this section:

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

128 Iterative Inorder Traversal Medium

Traverse a binary tree inorder without recursion.

Approach. Explicit stack: push the entire left spine, visit, then move right.

```
vector<int> inorder(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        out.push_back(cur->val);
        cur = cur->right;
    }
    return out;
}
```

Complexity: $O(n)$ time, $O(h)$ space.

129 Level Order Traversal Medium

Return node values grouped by depth.

Approach. BFS with a queue, processing one full level per outer iteration.

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> out;
    if (!root) return out;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        while (sz-->0) {
            TreeNode* n = q.front(); q.pop();
            level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        out.push_back(level);
    }
    return out;
}
```

Complexity: $O(n)$ time, $O(\text{width})$ space.

130 Maximum Depth Easy

Return the height of a binary tree.

Approach. One plus the larger of the two subtree depths.

```
int maxDepth(TreeNode* root) {
    return root ? 1 + max(maxDepth(root->left), maxDepth(root->right)) : 0;
}
```

Complexity: $O(n)$ time, $O(h)$ space.

131 Validate BST Medium

Check the binary search tree ordering property.

Approach. Carry an open interval down the recursion. Comparing only against the parent is the classic wrong answer.

```
bool validate(TreeNode* n, long long lo, long long hi) {
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return validate(n->left, lo, n->val) && validate(n->right, n->val, hi);
}

bool isValidBST(TreeNode* root) {
    return validate(root, LLONG_MIN, LLONG_MAX);
}
```

Complexity: $O(n)$ time, $O(h)$ space.

132 Lowest Common Ancestor in a BST Easy

Find the LCA of two nodes in a BST.

Approach. Descend while both targets lie on the same side; the split point is the LCA.

```
TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;
    }
    return nullptr;
}
```

Complexity: O(h) time, O(1) space.

133 Lowest Common Ancestor in a Binary Tree Medium

Find the LCA in a general binary tree.

Approach. Post-order: a node is the LCA if the two targets are found in different subtrees.

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* L = lca(root->left, p, q);
    TreeNode* R = lca(root->right, p, q);
    if (L && R) return root;
    return L ? L : R;
}
```

Complexity: O(n) time, O(h) space.

134 Invert Binary Tree Easy

Mirror a binary tree.

Approach. Swap children at every node.

```
TreeNode* invertTree(TreeNode* root) {
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

Complexity: O(n) time, O(h) space.

135 Symmetric Tree Easy

Is the tree a mirror of itself?

Approach. Compare left-left against right-right and left-right against right-left.

```
bool mirror(TreeNode* a, TreeNode* b) {
    if (!a && !b) return true;
    if (!a || !b || a->val != b->val) return false;
    return mirror(a->left, b->right) && mirror(a->right, b->left);
}
bool isSymmetric(TreeNode* root) {
    return !root || mirror(root->left, root->right);
}
```

Complexity: O(n) time, O(h) space.

136 Path Sum Easy

Does a root-to-leaf path sum to a target?

Approach. Subtract as you descend and test at leaves.

```
bool hasPathSum(TreeNode* root, int target) {
    if (!root) return false;
    if (!root->left && !root->right) return target == root->val;
    return hasPathSum(root->left, target - root->val) ||
           hasPathSum(root->right, target - root->val);
}
```

Complexity: O(n) time, O(h) space.

137 Diameter of a Binary Tree Easy

Longest path between any two nodes.

Approach. Compute height while updating a global best with left height plus right height at each node.

```
int diameterHelper(TreeNode* n, int& best) {
    if (!n) return 0;
    int L = diameterHelper(n->left, best);
    int R = diameterHelper(n->right, best);
    best = max(best, L + R);
    return 1 + max(L, R);
}
int diameterOfBinaryTree(TreeNode* root) {
    int best = 0;
    diameterHelper(root, best);
    return best;
}
```

Complexity: $O(n)$ time, $O(h)$ space.

138 Balanced Binary Tree Easy

Is every node's subtree heights within one?

Approach. Return -1 as a sentinel for imbalance to keep it a single pass.

```
int heightOrFail(TreeNode* n) {
    if (!n) return 0;
    int L = heightOrFail(n->left);
    if (L < 0) return -1;
    int R = heightOrFail(n->right);
    if (R < 0 || abs(L - R) > 1) return -1;
    return 1 + max(L, R);
}
bool isBalanced(TreeNode* root) { return heightOrFail(root) >= 0; }
```

Complexity: $O(n)$ time, $O(h)$ space.

139 BST Insert and Delete Medium

Insert into and delete from a binary search tree.

Approach. Delete has three cases; for two children, splice in the inorder successor.

```
TreeNode* insertBST(TreeNode* root, int v) {
    if (!root) return new TreeNode(v);
    if (v < root->val) root->left = insertBST(root->left, v);
    else root->right = insertBST(root->right, v);
    return root;
}
TreeNode* deleteBST(TreeNode* root, int v) {
    if (!root) return nullptr;
    if (v < root->val) root->left = deleteBST(root->left, v);
    else if (v > root->val) root->right = deleteBST(root->right, v);
    else {
        if (!root->left) return root->right;
        if (!root->right) return root->left;
        TreeNode* s = root->right;
        while (s->left) s = s->left; // inorder successor
        root->val = s->val;
        root->right = deleteBST(root->right, s->val);
    }
    return root;
}
```

Complexity: $O(h)$ time, $O(h)$ space.

140 Kth Smallest Element in a BST Medium

Return the kth smallest value.

Approach. Inorder traversal is sorted; stop after k pops.

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        if (--k == 0) return cur->val;
        cur = cur->right;
    }
    return -1;
}
```

Complexity: $O(h + k)$ time, $O(h)$ space.

Heaps and Priority Queues

Top-K and streaming-median patterns. Recognising that a problem wants a heap rather than a sort is usually the whole insight.

141 Kth Largest Element in a Stream Easy

Maintain the kth largest value as elements arrive.

Approach. Min-heap capped at size k; its top is always the kth largest.

```
class KthLargest {
    priority_queue<int, vector<int>, greater<int>> pq;
    int k;
public:
    KthLargest(int k_, vector<int>& nums) : k(k_) {
        for (int v : nums) add(v);
    }
    int add(int v) {
        pq.push(v);
        if ((int)pq.size() > k) pq.pop();
        return pq.top();
    }
};
```

Complexity: $O(\log k)$ per insertion, $O(k)$ space.

142 Find Median from Data Stream Hard

Report the running median of a stream.

Approach. Two heaps: a max-heap for the lower half, a min-heap for the upper, kept balanced within one element.

```
class MedianFinder {
    priority_queue<int> lo; // max-heap
    priority_queue<int, vector<int>, greater<int>> hi; // min-heap
public:
    void addNum(int v) {
        lo.push(v);
        hi.push(lo.top()); lo.pop();
        if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
    }
    double findMedian() {
        return lo.size() > hi.size() ? lo.top() : (lo.top() + hi.top()) / 2.0;
    }
};
```

Complexity: $O(\log n)$ per insertion, $O(n)$ space.

143 K Closest Points to Origin Medium

Return the k points nearest the origin.

Approach. Max-heap of size k on squared distance; no square root needed.

```
vector<vector<int>> kClosest(vector<vector<int>>& pts, int k) {
    priority_queue<pair<int,int>> pq; // (dist2, index)
    for (int i = 0; i < (int)pts.size(); i++) {
        int d = pts[i][0]*pts[i][0] + pts[i][1]*pts[i][1];
        pq.push({d, i});
        if ((int)pq.size() > k) pq.pop();
    }
    vector<vector<int>> out;
    while (!pq.empty()) { out.push_back(pts[pq.top().second]); pq.pop(); }
    return out;
}
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

144 Task Scheduler Medium

Minimum time to run tasks with a cooldown between identical tasks.

Approach. The most frequent task sets the skeleton; compute idle slots arithmetically rather than simulating.

```
int leastInterval(const vector<char>& tasks, int n) {
    vector<int> cnt(26, 0);
    for (char c : tasks) cnt[c - 'A']++;
    int mx = *max_element(cnt.begin(), cnt.end());
    int numMax = count(cnt.begin(), cnt.end(), mx);
    int slots = (mx - 1) * (n + 1) + numMax;
    return max((int)tasks.size(), slots);
}
```

Complexity: $O(n)$ time, $O(1)$ space.

Hardware connection. Structurally identical to scheduling operations under a resource hazard -- the same reasoning as your slot-1 replay.

145 Reorganize String Medium

Rearrange a string so no two adjacent characters match.

Approach. Greedily place the most frequent remaining character that differs from the previous one.

```
string reorganizeString(const string& s) {
    vector<int> cnt(26, 0);
    for (char c : s) cnt[c - 'a']++;
    priority_queue<pair<int, char>> pq;
    for (int i = 0; i < 26; i++) if (cnt[i]) pq.push({cnt[i], (char)('a' + i)});
    string out;
    pair<int, char> prev = {0, '#'};
    while (!pq.empty()) {
        auto cur = pq.top(); pq.pop();
        out += cur.second;
        if (prev.first > 0) pq.push(prev);
        cur.first--;
        prev = cur;
    }
    return out.size() == s.size() ? out : "";
}
```

Complexity: $O(n \log 26)$ time, $O(1)$ space.

146 Minimum Cost to Connect Ropes Easy

Repeatedly join the two shortest ropes; minimise total cost.

Approach. Min-heap; always merge the two smallest, which is Huffman coding.

```
int connectRopes(const vector<int>& r) {
    priority_queue<int, vector<int>, greater<int>> pq(r.begin(), r.end());
    int total = 0;
    while (pq.size() > 1) {
        int a = pq.top(); pq.pop();
        int b = pq.top(); pq.pop();
        total += a + b;
        pq.push(a + b);
    }
    return total;
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

147 Merge K Sorted Arrays Medium

Merge k sorted arrays into one sorted output.

Approach. Heap of (value, array index, element index) triples, k entries at a time.

```
vector<int> mergeKArrays(const vector<vector<int>>& arrs) {
    typedef tuple<int, int, int> T; // (value, which array, index)
    priority_queue<T, vector<T>, greater<T>> pq;
    for (int i = 0; i < (int)arrs.size(); i++)
        if (!arrs[i].empty()) pq.push({arrs[i][0], i, 0});
    vector<int> out;
    while (!pq.empty()) {
        int v, a, j;
        tie(v, a, j) = pq.top(); pq.pop();
        out.push_back(v);
        if (j + 1 < (int)arrs[a].size()) pq.push({arrs[a][j+1], a, j+1});
    }
    return out;
}
```

Complexity: $O(N \log k)$ time, $O(k)$ space.

Recursion and Backtracking

Enumerate, prune, undo. The template is identical across every problem here; learn it once and the variants become mechanical.

148 Subsets Medium

Generate the power set.

Approach. Backtracking: at each index choose to include or exclude, recording the state at every node.

```
void subsetsHelper(const vector<int>& a, int i, vector<int>& cur,
                  vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(cur); return; }
    subsetsHelper(a, i + 1, cur, out);           // exclude
    cur.push_back(a[i]);
    subsetsHelper(a, i + 1, cur, out);           // include
    cur.pop_back();                             // undo
}
```

Complexity: $O(n \cdot 2^n)$ time and space.

149 Permutations Medium

Generate all orderings of distinct integers.

Approach. Swap element i with each candidate, recurse, then swap back.

```
void permuteHelper(vector<int>& a, int i, vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(a); return; }
    for (int j = i; j < (int)a.size(); j++) {
        swap(a[i], a[j]);
        permuteHelper(a, i + 1, out);
        swap(a[i], a[j]);
    }
}
```

Complexity: $O(n \cdot n!)$ time, $O(n)$ recursion depth.

150 Combination Sum Medium

All combinations summing to a target, values reusable.

Approach. Recurse allowing the same index again; prune when the remainder goes negative.

```
void combo(const vector<int>& a, int i, int rem, vector<int>& cur,
           vector<vector<int>>& out) {
    if (rem == 0) { out.push_back(cur); return; }
    if (rem < 0 || i == (int)a.size()) return;
    cur.push_back(a[i]);
    combo(a, i, rem - a[i], cur, out);           // reuse allowed
    cur.pop_back();
    combo(a, i + 1, rem, cur, out);
}
```

Complexity: Exponential time, $O(\text{target})$ depth.

151 N-Queens Hard

Place n queens on an $n \times n$ board with no mutual attacks.

Approach. Place one queen per row; track occupied columns and both diagonals with boolean arrays or bitmasks.

```
void solveNQ(int n, int row, vector<bool>& col, vector<bool>& d1,
             vector<bool>& d2, vector<int>& pos, int& count) {
    if (row == n) { count++; return; }
    for (int c = 0; c < n; c++) {
        int i1 = row + c, i2 = row - c + n;
        if (col[c] || d1[i1] || d2[i2]) continue;
        col[c] = d1[i1] = d2[i2] = true;
        pos.push_back(c);
        solveNQ(n, row + 1, col, d1, d2, pos, count);
        pos.pop_back();
        col[c] = d1[i1] = d2[i2] = false;
    }
}
```

Complexity: $O(n!)$ worst case, $O(n)$ space.

Hardware connection. The bitmask variant of this is a favourite follow-up: columns and diagonals become three integers.

152 Generate Parentheses Medium

Generate all valid combinations of n bracket pairs.

Approach. Track counts of open and closed; only close when it keeps the prefix valid.

```
void genParen(int open, int close, string& cur, vector<string>& out) {
    if (open == 0 && close == 0) { out.push_back(cur); return; }
    if (open > 0) { cur += '('; genParen(open - 1, close, cur, out); cur.pop_back(); }
    if (close > open) { cur += ')'; genParen(open, close - 1, cur, out); cur.pop_back(); }
}
```

Complexity: $O(\text{Catalan}(n))$ time, $O(n)$ depth.

153 Word Search Medium

Does a word exist along adjacent cells of a grid?

Approach. DFS from each cell, marking visited in place and restoring on backtrack.

```
bool dfsWord(vector<vector<char>>& g, const string& w, int i, int j, int k) {
    if (k == (int)w.size()) return true;
    if (i < 0 || j < 0 || i >= (int)g.size() || j >= (int)g[0].size()) return false;
    if (g[i][j] != w[k]) return false;
    char save = g[i][j];
    g[i][j] = '#';
    bool found = dfsWord(g, w, i+1, j, k+1) || dfsWord(g, w, i-1, j, k+1) ||
                 dfsWord(g, w, i, j+1, k+1) || dfsWord(g, w, i, j-1, k+1);
    g[i][j] = save;
    return found;
}
```

Complexity: $O(mn * 4^L)$ time, $O(L)$ depth.

154 Palindrome Partitioning Medium

Split a string so every part is a palindrome.

Approach. Try each prefix; recurse on the remainder only when the prefix is a palindrome.

```
bool isPal(const string& s, int i, int j) {
    while (i < j) if (s[i++] != s[j--]) return false;
    return true;
}
void partHelper(const string& s, int i, vector<string>& cur,
               vector<vector<string>>& out) {
    if (i == (int)s.size()) { out.push_back(cur); return; }
    for (int j = i; j < (int)s.size(); j++)
        if (isPal(s, i, j)) {
            cur.push_back(s.substr(i, j - i + 1));
            partHelper(s, j + 1, cur, out);
            cur.pop_back();
        }
}
```

Complexity: $O(n * 2^n)$ time, $O(n)$ depth.

Graphs

Pay special attention to topological sort, cycle detection, and longest path in a DAG. These are not abstract exercises for you -- they are what static timing analysis, combinational loop detection, and netlist traversal actually do. Being able to draw that connection in a Google Silicon interview is worth more than solving the problem quickly.

155 Number of Islands Medium

Count connected regions of land in a grid.

Approach. Flood-fill from every unvisited land cell, sinking the region as you go.

```
void sink(vector<vector<char>>& g, int i, int j) {
    if (i < 0 || j < 0 || i >= (int)g.size() || j >= (int)g[0].size() || g[i][j] != '1') return;
    g[i][j] = '0';
    sink(g, i+1, j); sink(g, i-1, j); sink(g, i, j+1); sink(g, i, j-1);
}

int numIslands(vector<vector<char>>& g) {
    int c = 0;
    for (int i = 0; i < (int)g.size(); i++)
        for (int j = 0; j < (int)g[0].size(); j++)
            if (g[i][j] == '1') { c++; sink(g, i, j); }
    return c;
}
```

Complexity: $O(mn)$ time, $O(mn)$ worst-case depth.

156 Topological Sort (Kahn's Algorithm) Medium

Order a DAG so every edge points forward.

Approach. Repeatedly emit nodes of in-degree zero, decrementing successors. If fewer than n nodes are emitted, a cycle exists.

```
vector<int> topoSort(int n, const vector<vector<int>>& adj) {
    vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    for (int u = 0; u < n; u++) if (!indeg[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return (int)order.size() == n ? order : vector<int>{}; // empty => cycle
}
```

Complexity: $O(V+E)$ time, $O(V)$ space.

Hardware connection. This is levelisation. An EDA tool topologically sorts the netlist before evaluating it, and the empty-result case is exactly a combinational loop error.

157 Detect Cycle in a Directed Graph Medium

Determine whether a directed graph has a cycle.

Approach. DFS with three colours: white unvisited, grey on the current stack, black finished. A grey edge is a back edge.

```
bool dfsCycle(int u, const vector<vector<int>>& adj, vector<int>& color) {
    color[u] = 1; // grey
    for (int v : adj[u]) {
        if (color[v] == 1) return true; // back edge
        if (color[v] == 0 && dfsCycle(v, adj, color)) return true;
    }
    color[u] = 2; // black
    return false;
}
```

Complexity: $O(V+E)$ time, $O(V)$ space.

Hardware connection. Combinational loop detection in synthesis is this algorithm run over the gate graph.

158 Longest Path in a DAG Hard

Find the longest path in a weighted DAG.

Approach. Relax edges in topological order; each node's longest path is the best incoming path plus that edge.

```
vector<long long> longestPath(int n, const vector<vector<pair<int,int>>& adj,
                             const vector<int>& order) {
    vector<long long> dist(n, 0);
    for (int u : order)
        for (auto& e : adj[u])
            dist[e.first] = max(dist[e.first], dist[u] + e.second);
    return dist;
}
```

Complexity: $O(V+E)$ time, $O(V)$ space.

Hardware connection. This IS static timing analysis. Nodes are pins, edge weights are cell and net delays, and the longest path is the critical path. If asked this at Google, say so.

159 Course Schedule Medium

Can all courses be finished given prerequisites?

Approach. Exactly cycle detection on a directed graph; reuse Kahn's algorithm.

```
bool canFinish(int n, const vector<pair<int,int>>& pre) {
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (auto& p : pre) { adj[p.second].push_back(p.first); indeg[p.first]++; }
    queue<int> q;
    for (int i = 0; i < n; i++) if (!indeg[i]) q.push(i);
    int seen = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); seen++;
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return seen == n;
}
```

Complexity: $O(V+E)$ time, $O(V+E)$ space.

160 Clone Graph Medium

Deep-copy an undirected graph.

Approach. DFS or BFS with a map from original node to clone, created before recursing to handle cycles.

```
struct GNode { int val; vector<GNode*> nb; };
GNode* cloneGraph(GNode* n, unordered_map<GNode*, GNode*>& seen) {
    if (!n) return nullptr;
    auto it = seen.find(n);
    if (it != seen.end()) return it->second;
    GNode* copy = new GNode{n->val, {}};
    seen[n] = copy;
    for (GNode* x : n->nb) copy->nb.push_back(cloneGraph(x, seen));
    return copy;
}
```

Complexity: $O(V+E)$ time, $O(V)$ space.

161 Union-Find (Disjoint Set Union) Medium

Support union and connectivity queries.

Approach. Path compression plus union by rank gives near-constant amortised cost.

```
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { for (int i = 0; i < n; i++) p[i] = i; }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) r[a]++;
        return true;
    }
};
```

Complexity: Near $O(1)$ amortised per operation, $O(n)$ space.

Hardware connection. Used in physical design for net grouping and in equivalence checking.

162 Bipartite Check Medium

Can the graph be two-coloured?

Approach. BFS colouring; a conflicting edge means an odd cycle exists.

```
bool isBipartite(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> color(n, -1);
    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        queue<int> q; q.push(s); color[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) { color[v] = color[u] ^ 1; q.push(v); }
                else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}
```

Complexity: $O(V+E)$ time, $O(V)$ space.

163 Dijkstra's Shortest Path Medium

Shortest paths from a source with non-negative weights.

Approach. Min-heap of (distance, node); skip stale entries rather than decreasing keys.

```
vector<long long> dijkstra(int n, const vector<vector<pair<int,int>>>& adj, int src) {
    const long long INF = (1LL << 60);
    vector<long long> dist(n, INF);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>,
        greater<pair<long long,int>>> pq;

    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto top = pq.top(); pq.pop();
        long long d = top.first;
        int u = top.second;
        if (d > dist[u]) continue; // stale
        for (auto& e : adj[u])
            if (d + e.second < dist[e.first]) {
                dist[e.first] = d + e.second;
                pq.push({dist[e.first], e.first});
            }
    }
    return dist;
}
```

Complexity: $O((V+E) \log V)$ time, $O(V)$ space.

164 Rotting Oranges (Multi-source BFS) Medium

Time for rot to spread to every fresh orange.

Approach. Seed the queue with all rotten cells at once and BFS level by level.

```
int orangesRotting(vector<vector<int>>& g) {
    queue<pair<int,int>> q;
    int fresh = 0, R = g.size(), C = g[0].size();
    for (int i = 0; i < R; i++) for (int j = 0; j < C; j++) {
        if (g[i][j] == 2) q.push({i, j});
        else if (g[i][j] == 1) fresh++;
    }
    int t = 0, dx[] = {1,-1,0,0}, dy[] = {0,0,1,-1};
    while (!q.empty() && fresh) {
        int sz = q.size();
        while (sz--) {
            auto cell = q.front(); q.pop();
            for (int d = 0; d < 4; d++) {
                int x = cell.first + dx[d], y = cell.second + dy[d];
                if (x < 0 || y < 0 || x >= R || y >= C || g[x][y] != 1) continue;
                g[x][y] = 2; fresh--;
                q.push({x, y});
            }
        }
        t++;
    }
    return fresh ? -1 : t;
}
```

Complexity: $O(mn)$ time, $O(mn)$ space.

Dynamic Programming

Google may ask; Nvidia and TI rarely do. Learn the classic set well enough to recognise the pattern and write the recurrence -- do not grind this at the expense of bit manipulation.

165 Climbing Stairs Easy

Count ways to climb n stairs taking 1 or 2 steps.

Approach. Fibonacci recurrence; keep two rolling variables instead of a table.

```
int climbStairs(int n) {
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

166 House Robber Medium

Maximise the sum of non-adjacent elements.

Approach. At each house choose to rob it plus the best two back, or skip to the best one back.

```
int rob(const vector<int>& a) {
    int prev = 0, cur = 0;
    for (int v : a) { int t = max(cur, prev + v); prev = cur; cur = t; }
    return cur;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

167 Coin Change Medium

Fewest coins summing to an amount.

Approach. Unbounded knapsack: $dp[x]$ is the best count for amount x, relaxed over every coin.

```
int coinChange(const vector<int>& coins, int amount) {
    const int INF = 1e9;
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++)
        for (int c : coins)
            if (c <= x) dp[x] = min(dp[x], dp[x - c] + 1);
    return dp[amount] >= INF ? -1 : dp[amount];
}
```

Complexity: $O(\text{amount} * \text{coins})$ time, $O(\text{amount})$ space.

168 Longest Increasing Subsequence Medium

Length of the longest strictly increasing subsequence.

Approach. Patience sorting: keep the smallest tail per length and binary search the insertion point.

```
int lengthOfLIS(const vector<int>& a) {
    vector<int> tails;
    for (int v : a) {
        auto it = lower_bound(tails.begin(), tails.end(), v);
        if (it == tails.end()) tails.push_back(v);
        else *it = v;
    }
    return tails.size();
}
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

169 Longest Common Subsequence Medium

Length of the LCS of two strings.

Approach. Two-dimensional table: match extends the diagonal, mismatch takes the better neighbour.

```
int lcs(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
            : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

Complexity: $O(nm)$ time, $O(nm)$ space (reducible to $O(m)$).

170 Edit Distance Hard

Minimum insertions, deletions, and replacements to transform one string into another.

Approach. Three-way minimum over insert, delete, and replace, with a free diagonal on a match.

```
int minDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1]
                : 1 + min(dp[i-1][j-1], min(dp[i-1][j], dp[i][j-1]));
    return dp[n][m];
}
```

Complexity: $O(nm)$ time, $O(nm)$ space.

171 0/1 Knapsack Medium

Maximise value within a weight capacity, each item used at most once.

Approach. Iterate capacity downward so each item is considered only once per row.

```
int knapsack(const vector<int>& w, const vector<int>& v, int cap) {
    vector<int> dp(cap + 1, 0);
    for (int i = 0; i < (int)w.size(); i++)
        for (int c = cap; c >= w[i]; c--)
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[cap];
}
```

Complexity: $O(n * \text{cap})$ time, $O(\text{cap})$ space.

172 Unique Paths Medium

Count monotone lattice paths across a grid.

Approach. Each cell is the sum of the cell above and the cell to the left; one rolling row suffices.

```
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[j] += dp[j-1];
    return dp[n-1];
}
```

Complexity: $O(mn)$ time, $O(n)$ space.

173 Word Break Medium

Can a string be segmented into dictionary words?

Approach. $dp[i]$ is true if some split point j has $dp[j]$ true and $s[j..i]$ in the dictionary.

```
bool wordBreak(const string& s, const vector<string>& dict) {
    unordered_set<string> d(dict.begin(), dict.end());
    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j < i; j++)
            if (dp[j] && d.count(s.substr(j, i - j))) { dp[i] = true; break; }
    return dp[n];
}
```

Complexity: $O(n^2 * L)$ time, $O(n)$ space.

174 Maximum Product Subarray Medium

Largest product of a contiguous subarray.

Approach. Track both the maximum and minimum ending here, since a negative flips their roles.

```
int maxProduct(const vector<int>& a) {
    int best = a[0], hi = a[0], lo = a[0];
    for (size_t i = 1; i < a.size(); i++) {
        int v = a[i];
        if (v < 0) swap(hi, lo);
        hi = max(v, hi * v);
        lo = min(v, lo * v);
        best = max(best, hi);
    }
    return best;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

Hardware Crossover Problems

The highest-leverage section in this document. Each problem here has a direct RTL analogue you have already built. Solving them is table stakes; being able to describe the hardware implementation alongside the software one is what separates you from a generic candidate.

175 LRU Cache Medium

Design a fixed-capacity cache with $O(1)$ get and put, evicting the least recently used entry.

Approach. Hash map from key to list iterator, plus a doubly linked list ordered by recency. Any access splices the node to the front; eviction pops the back.

```
class LRUCache {
    int cap;
    list<pair<int,int>> dll;
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int c) : cap(c) {}
    int get(int key) {
        auto it = mp.find(key);
        if (it == mp.end()) return -1;
        dll.splice(dll.begin(), dll, it->second);
        return it->second->second;
    }
    void put(int key, int val) {
        auto it = mp.find(key);
        if (it != mp.end()) {
            it->second->second = val;
            dll.splice(dll.begin(), dll, it->second);
            return;
        }
        if ((int)dll.size() == cap) {
            mp.erase(dll.back().first);
            dll.pop_back();
        }
        dll.push_front({key, val});
        mp[key] = dll.begin();
    }
};
```

Complexity: $O(1)$ per operation, $O(\text{capacity})$ space.

Hardware connection. Treat this as mandatory. Expect the follow-up 'how would you build this in hardware?' -- the answer is that true LRU needs age bits or a matrix of comparators per set, which is why real caches use pseudo-LRU (tree-PLRU) or random replacement. You implemented a direct-mapped cache, so you can contrast the two directly.

176 Design HashMap Easy

Implement a hash map without library containers.

Approach. Fixed bucket array with chaining. Choosing a power-of-two bucket count lets you mask instead of divide.

```
class MyHashMap {
    static const int NB = 1024;
    vector<list<pair<int,int>>> b;
public:
    MyHashMap() : b(NB) {}
    int idx(int k) const { return k & (NB - 1); } // mask, not modulo
    void put(int k, int v) {
        for (auto& kv : b[idx(k)]) if (kv.first == k) { kv.second = v; return; }
        b[idx(k)].push_back({k, v});
    }
    int get(int k) {
        for (auto& kv : b[idx(k)]) if (kv.first == k) return kv.second;
        return -1;
    }
    void remove(int k) {
        auto& lst = b[idx(k)];
        for (auto it = lst.begin(); it != lst.end(); ++it)
            if (it->first == k) { lst.erase(it); return; }
    }
};
```

Complexity: $O(1)$ average per operation, $O(\text{NB} + n)$ space.

Hardware connection. Masking instead of modulo is exactly why cache index fields are bit slices of the address.

177 Direct-Mapped Cache Simulator Medium

Given a stream of addresses, report hits and misses for a direct-mapped cache.

Approach. Split the address into tag, index, and offset by bit position. Compare the stored tag at the index and check the valid bit.

```
struct DirectMappedCache {
    int idxBits, offBits;
    vector<uint32_t> tag;
    vector<bool> valid;
    int hits = 0, misses = 0;
    DirectMappedCache(int lines, int blockBytes) {
        idxBits = log2floor(lines);
        offBits = log2floor(blockBytes);
        tag.assign(lines, 0);
        valid.assign(lines, false);
    }
    void access(uint32_t addr) {
        uint32_t index = (addr >> offBits) & ((1u << idxBits) - 1);
        uint32_t t = addr >> (offBits + idxBits);
        if (valid[index] && tag[index] == t) hits++;
        else { misses++; valid[index] = true; tag[index] = t; }
    }
};
```

Complexity: O(1) per access, O(lines) space.

Hardware connection. This is your Verilog cache written in C. Expect questions on conflict misses, why associativity helps, and how the write-back/write-allocate policy you implemented changes the miss handling.

178 Round-Robin Arbiter Medium

Given a bitmask of requests and the last grant, produce the next grant fairly.

Approach. Rotate the request mask so the position after the last grant becomes bit 0, take the lowest set bit, then rotate back.

```
// Returns a one-hot grant mask, or 0 if no requests.
uint32_t roundRobinGrant(uint32_t req, int lastGrant, int width) {
    if (!req) return 0;
    int start = (lastGrant + 1) % width;
    uint32_t mask = (width == 32) ? 0xFFFFFFFFu : ((1u << width) - 1);
    uint32_t rot = ((req >> start) | (req << (width - start))) & mask;
    uint32_t low = rot & (~rot + 1); // isolate lowest set bit
    int pos = log2floor(low);
    return 1u << ((pos + start) % width);
}
```

Complexity: O(1) time, O(1) space.

Hardware connection. A genuine RTL block. The software rotate models a barrel shifter; the lowest-set-bit isolation is the priority encoder. Mention that a real implementation uses a mask-and-priority-encode pair to stay single-cycle.

179 Priority Encoder and One-Hot Conversion Easy

Convert a request vector to the index of its highest-priority set bit, and back to one-hot.

Approach. Lowest set bit for LSB-priority, MSB smear for MSB-priority. One-hot is a single shift.

```
int priorityEncodeLSB(uint32_t req) { // -1 if none
    if (!req) return -1;
    return log2floor(req & (~req + 1));
}
int priorityEncodeMSB(uint32_t req) {
    if (!req) return -1;
    return log2floor(req);
}
uint32_t toOneHot(int idx) { return idx < 0 ? 0u : (1u << idx); }
```

Complexity: O(1) time, O(1) space.

Hardware connection. Straight out of your interrupt-controller and arbiter vocabulary.

180 LFSR Next-State and Sequence Medium

Compute the next state of a maximal-length Galois LFSR and generate its full sequence.

Approach. Shift right; if the bit shifted out is 1, XOR the polynomial mask into the state.

```
uint32_t lfsrNext(uint32_t state, uint32_t poly) {
    uint32_t lsb = state & 1u;
    state >>= 1;
    if (lsb) state ^= poly;           // e.g. 0xB400 for 16-bit maximal
    return state;
}
int lfsrPeriod(uint32_t seed, uint32_t poly) {
    uint32_t s = seed;
    int n = 0;
    do { s = lfsrNext(s, poly); n++; } while (s != seed);
    return n;                        // 2^k - 1 for a maximal polynomial
}
```

Complexity: $O(1)$ per step; $O(2^k)$ to measure the period.

Hardware connection. You already built LFSRs for your juniors' assignments. Expect 'why $2^k - 1$ and not 2^k ' -- the all-zero state is an absorbing state and is excluded from the cycle.

181 Barrel Shifter Simulation Medium

Implement rotate-left and rotate-right for a fixed width without a loop.

Approach. Log-depth multiplexer stages: shift by 16, 8, 4, 2, 1 conditionally on the bits of the shift amount.

```
uint32_t rotl32(uint32_t v, int n) {
    n &= 31;
    return n ? ((v << n) | (v >> (32 - n))) : v; // guard n==0: shift by 32 is UB
}
uint32_t rotr32(uint32_t v, int n) {
    n &= 31;
    return n ? ((v >> n) | (v << (32 - n))) : v;
}
```

Complexity: $O(1)$ time, $O(1)$ space.

Hardware connection. The undefined behaviour of shifting by the full width is a favourite C trap. In hardware the barrel shifter is five multiplexer stages, one per bit of the shift amount.

182 Bit-Reversal Permutation for FFT Medium

Reorder an array into bit-reversed index order.

Approach. For each index compute its bit-reversal within $\log_2(n)$ bits and swap once per pair.

```
void bitReversePermute(vector<int>& a) {
    int n = a.size(), bits = log2floor(n);
    for (int i = 0; i < n; i++) {
        int r = 0;
        for (int b = 0; b < bits; b++) if (i & (1 << b)) r |= 1 << (bits - 1 - b);
        if (i < r) swap(a[i], a[r]);
    }
}
```

Complexity: $O(n \log n)$ time, $O(1)$ space.

Hardware connection. The addressing pattern of a hardware FFT butterfly network -- in RTL the reversal is again just wiring.

183 Producer-Consumer with a Bounded Buffer Medium

Coordinate a producer and consumer over a fixed-size queue.

Approach. Mutex plus two condition variables. The full and empty predicates are the same ones your FIFO flags implement.

```
template <typename T>
class BoundedBuffer {
    queue<T> q; size_t cap;
    mutex m; condition_variable notFull, notEmpty;
public:
    BoundedBuffer(size_t c) : cap(c) {}
    void put(const T& v) {
        unique_lock<mutex> lk(m);
        notFull.wait(lk, [&]{ return q.size() < cap; });
        q.push(v);
        notEmpty.notify_one();
    }
    T take() {
        unique_lock<mutex> lk(m);
        notEmpty.wait(lk, [&]{ return !q.empty(); });
        T v = q.front(); q.pop();
        notFull.notify_one();
        return v;
    }
};
```

Complexity: O(1) per operation, O(capacity) space.

Hardware connection. The software expression of FIFO backpressure. Ready/valid handshaking is the hardware equivalent, and the wait predicates correspond directly to your full and empty flags.

184 Count Bits for 0..n (DP on Bits) Easy

Return the popcount of every integer from 0 to n.

Approach. $dp[i] = dp[i \gg 1] + (i \& 1)$: the count of i is the count of i without its low bit, plus that bit.

```
vector<int> countBits(int n) {
    vector<int> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) dp[i] = dp[i >> 1] + (i & 1);
    return dp;
}
```

Complexity: O(n) time, O(n) space.

Hardware connection. A neat bridge between the bit-manipulation and dynamic-programming sections.

C and C++ Fundamentals for Hardware Interviews

At TI especially, these get asked more often than algorithms. Nvidia uses them to check that you can be trusted near a driver or a testbench. Answers should be crisp.

A1. What does volatile mean, and when is it mandatory?

It tells the compiler that the variable may change outside the program's control, so every read must actually load from memory and reads cannot be cached in a register or reordered away. It is mandatory for memory-mapped hardware registers, variables modified in an ISR, and flags shared with another execution context. It is *not* a synchronisation primitive and does not make anything atomic.

A2. Difference between `const char* p`, `char* const p`, and `const char* const p`?

Read right to left. `const char* p` is a mutable pointer to constant data. `char* const p` is a constant pointer to mutable data. `const char* const p` is a constant pointer to constant data.

A3. What are the three meanings of static in C++?

At file scope it gives internal linkage, limiting the symbol to that translation unit. Inside a function it gives the variable static storage duration, so it persists across calls. Inside a class it makes the member shared by all instances rather than per-object.

A4. Explain struct padding and alignment.

Compilers insert padding so each member sits at an address that is a multiple of its alignment, and pad the struct itself so arrays stay aligned. Reordering members from largest to smallest usually shrinks the struct. This matters for register maps and anything crossing a hardware boundary, where you would use `#pragma pack` or explicit padding fields.

A5. What is a union, and what is the classic hardware use?

A union overlays all members in the same storage, so only one is valid at a time. The common embedded use is a register overlay: a 32-bit word aliased with a bitfield struct, letting you read the whole register or individual fields. Note that reading a member other than the last written one is technically undefined behaviour in C++, though it is widely relied upon in embedded C.

A6. How do you detect endianness at runtime?

Write a multi-byte value and inspect its lowest-addressed byte:

```
uint32_t x = 1; bool little = *(char*)&x == 1;
```

Endianness matters when a bus, a file format, or a protocol fixes a byte order that differs from the host.

A7. Access a hardware register at a fixed address.

```
#define REG (*(volatile uint32_t*)0x40001000u)
```

Then `REG |= (1u << 3);` sets bit 3. Note that this is a read-modify-write; on a register with write-1-to-clear bits it can clear unrelated status flags, which is a real bug source.

A8. What is undefined behaviour in a left shift?

Shifting by an amount greater than or equal to the operand width is undefined, as is shifting a signed value into or past the sign bit. Use unsigned types for bit manipulation and mask the shift amount.

A9. Difference between a pointer and a reference in C++?

A reference must be initialised, can never be rebound, and cannot be null. A pointer can be null, reassigned, and does arithmetic. Prefer references for parameters that are always valid.

A10. What is RAII?

Resource Acquisition Is Initialisation: bind a resource's lifetime to an object's scope so the destructor releases it automatically, even when an exception unwinds. This is what `unique_ptr`, `lock_guard`, and `fstream` all implement.

A11. When must a destructor be virtual?

Whenever an object may be deleted through a base-class pointer. Without it the derived destructor never runs and resources leak. Any class intended as a polymorphic base should have a virtual or protected destructor.

A12. How does a virtual function call work?

Each polymorphic class has a vtable of function pointers; each object stores a vptr to it. A virtual call loads the vptr, indexes the vtable, and calls indirectly. The cost is an extra load and an indirect branch, which is why virtual calls resist inlining.

A13. Stack versus heap: when does each get used?

Stack allocation is a pointer bump, automatically reclaimed, limited in size, and ideal for short-lived objects. Heap allocation is explicit, slower, and unbounded in lifetime. Prefer the stack, and prefer smart pointers when you must use the heap.

A14. What is a memory leak, and what is a dangling pointer?

A leak is allocated memory that is never freed and no longer reachable. A dangling pointer is a pointer to memory that has already been freed; dereferencing it is undefined behaviour. Setting pointers to `nullptr` after deletion and using smart pointers avoids most of both.

A15. What does inline actually do?

It is a hint to expand the function body at the call site, but its guaranteed effect is on linkage: it permits multiple identical definitions across translation units. Modern compilers decide inlining themselves largely irrespective of the keyword.

A16. Write a macro to swap two values, and state its hazards.

```
#define SWAP(a,b) do { typedef(a) _t = (a); (a) = (b); (b) = _t; } while (0)
```

Hazards: arguments are evaluated more than once, so side effects such as `SWAP(x, arr[i++])` misbehave; parenthesise every use; and the do-while wrapper is required so the macro works as a single statement inside an unbraced `if`.